

자연어처리와 컴퓨터비전 심층학습 10장 (비전 트랜스포머)

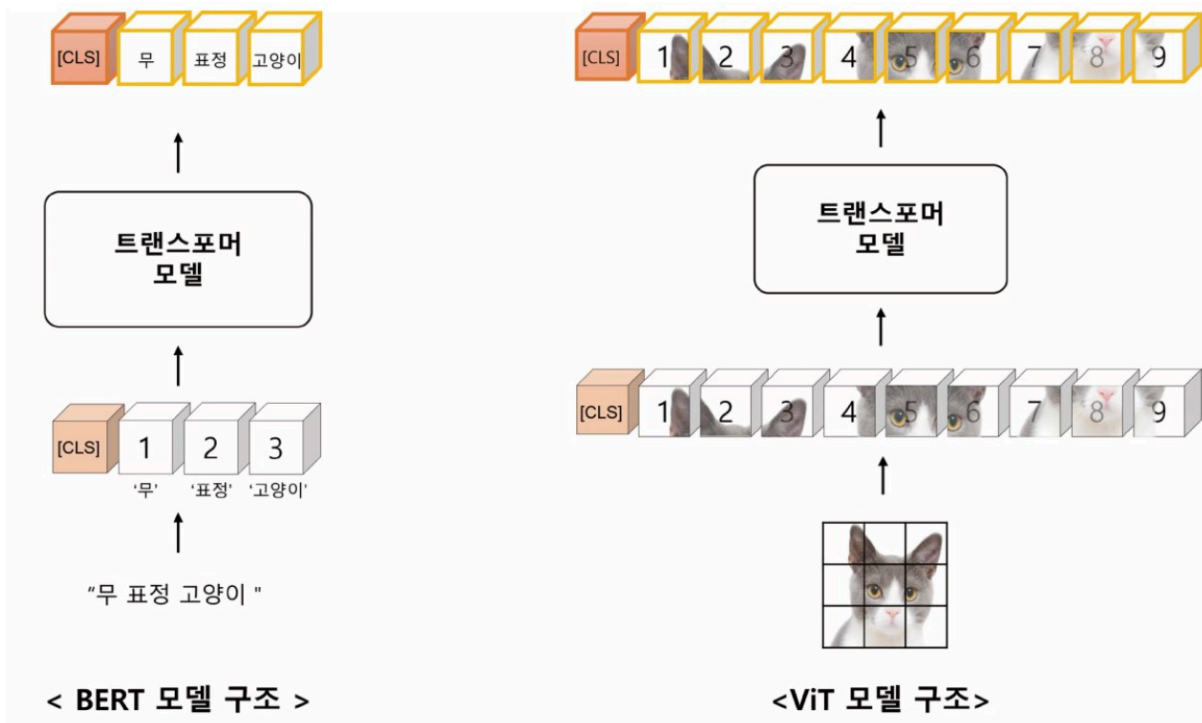
📄 제출	제출 전
📄 과제 유형	예습
📅 마감일	@2025년 5월 5일
☑ 발표	☐
☑ 완료	☐
📅 주차	9주차

- 2020년 구글의 An Image is Worth 16 × 16 Words: Transformers for Image Recognition at Scale 논문을 통해 소개된 이미지 인식을 위한 딥러닝 모델
- 이미지를 자연어 처리 방식처럼 분류해보려는 시도에 의해 탄생함
- CNN의 합성곱 계층 대신 트랜스포머 모델에서 사용되는 **셀프 어텐션**을 적용하여 **전체 이미지를 한 번에 처리**
- 이미지 패치가 왼 → 오, 위 → 아래 방향으로 순차적으로 입력되어 2차원 구조의 이미지 특성을 온전히 반영한다고 할 수 없음
- 이러한 문제를 해결하기 위해 물체의 크기나 해상도를 계층적으로 학습하는 **스윙 트랜스포머**와 **CvT (Convolutional Vision Transformer)** 모델이 제안됨
- 스윙 트랜스포머
 - **Local Window**를 활용해 각 계층의 어텐션이 되는 패치의 크기와 개수를 다양하게 구성해 이미지 특징을 학습시킴
 - ViT 구조의 기존 셀프 어텐션을 로컬 윈도우 안에 대한 어텐션, 로컬 윈도우 간의 어텐션으로 수행하여 이미지 특징을 계층적으로 학습
 - 이 어텐션 함수에는 상대적 위치 편향을 반영하여 어텐션 값 자체에 위치적 정보를 포함시킴
- CvT
 - 기존 합성곱 연산을 ViT에 적용한 모델

- 저수준 특징(low-level feature)과 고수준 특징(high-level feature)을 계층적으로 반영함
 - 사람 얼굴에서 저수준 특징: 눈, 코, 입 같은 작은 단위
 - 사람 얼굴에서 고수준 특징: 눈, 코, 입을 포함하는 전체 얼굴
- 어텐션 연산 과정에서 쿼리 (Query, Q), 키 (Key, K), 값 (Value, V) 중 키와 값을 기존 특징 벡터보다 축소해 계산 복잡도를 감소 시킴

ViT

- 자연어 처리: 트랜스포머 모델이 큰 성능 향상을 보임
- ViT: 트랜스포머 구조 자체를 컴퓨터 비전 분야에 적용한 첫 번째 연구



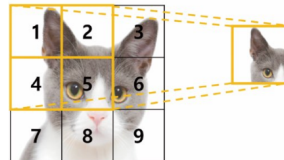
- BERT와 ViT 모델 모두 트랜스포머 모델의 구조를 사용하는, 자연어 처리에 널리 사용되는 모델이지만, 입력 데이터를 만드는 과정이 서로 다르다.
 - BERT: 문장보다 작은 단위의 토큰들이 순차적으로 입력 (자연어 처리)
 - ViT: 이미지가 격자로 작은 단위의 이미지 패치로 나뉘어 순차적으로 입력됨 ➡ 이 이미지 패치가 왼 → 오, 위 → 아래에서 표현된 시퀀스 배열을 가정함

합성곱 모델과 ViT 모델 비교

- 합성곱 신경망과 트랜스 포머 모두 이미지 특징을 잘 표현하는 임베딩을 만들고자함

합성곱 신경망 임베딩

- 이미지 패치 중 일부만 선택하여 학습 ➡ 이미지 전체 특징 추출
 - 고양이 이미지에서 왼쪽 눈의 특징을 추출하고자 하면, 합성곱 신경망은 이미지에서 해당 부분만 선택해 학습함

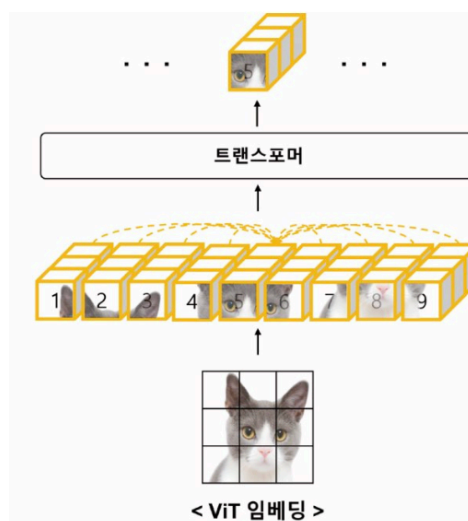


< CNN 임베딩 >

- 1, 2, 4, 5 이미지 패치만 학습되고 나머지 이미지 패치는 학습에 관여 x
- 좁은 수용 영역(Receptive field, RF)을 가진 합성곱 신경망은 전체 이미지 정보를 표현하는데 수많은 계층이 필요하다

ViT 임베딩

- 이미지를 작은 패치들로 나눠 각 패치 간의 상관 관계를 학습함. 이를 위해 **셀프 어텐션 방법**을 사용해 모든 이미지 패치가 서로에게 주는 영향을 고려해 이미지의 전체 특징을 추출함
 - 모든 이미지 패치가 학습에 관여



< ViT 임베딩 >

- 패치 단위로 이미지를 처리해서 더 작은 모델로도 높은 성능을 얻을 수 있음

- **어텐션 거리**(쿼리 벡터와 값 벡터 사이의 유사도 (거리) 를 내적으로 계산한 것)를 계산하여 오직 한 개의 레이어로 전체 이미지 정보를 표현할 수 있음
- 입력 이미지 크기가 고정되어 있어 크기가 다른 이미지를 처리하면 이미지 크기를 맞추는 전처리가 필요하며, 패치 간의 상대적 위치 정보만 고려하기 때문에 이미지 변환에 취약할 수 있음

ViT의 귀납적 편향

- 딥러닝 모델의 **귀납적 편향(Inductive Bias)**은 일반화 성능 향상을 위한 모델의 가정 (Assumption)을 의미한다.
 - Ex. 이미지 데이터는 공간적 관계를 잘 표현하므로 지역적 편향을 가진 합성곱 신경망 모델이 많이 사용됨. 합성곱 신경망은 이미지를 패치화하여 전체 이미지 특징을 추론하기 때문에 (지역적 특징을 강조) 이미지 데이터에서 좋은 성능을 발휘함
 - Ex. 시계열 데이터는 시간적 관계를 잘 표현하므로, 시퀀셜 편향을 가진 순환 신경망 모델이 많이 사용됨. 순환 신경망은 이전 시간 상태를 기억하고, 현재 입력과 결합하여 다음 상태를 예측하기 때문에 (순차적 특징을 강조) 시계열 데이터에서 좋은 성능을 발휘함
- 하지만, 이런 특정 관계 편향이 강할 수록, 다른 다양한 관계를 표현하기 어려워 귀납적 편향이 약한 모델을 선호하게 됨
- ➡ **ViT 모델**은 입력 데이터의 다양한 Query, Key, Value의 임베딩 형태로 일반화된 관계를 학습하기 때문에 **귀납적 편향이 거의 없고**, 대용량 데이터에 대해서 이미지 특징들의 관계를 잘 학습함

표 10.1 딥러닝 모델의 귀납적 편향

딥러닝 모델	귀납적 편향
합성곱 신경망(CNN)	지역적 편향
순환 신경망(RNN)	순차적 편향
ViT	귀납적 편향이 거의 없음

- 귀납적 편향은 해당 모델이 가지는 구조와 매개변수들이 **데이터에 적합한 가정**을 하고 있음을 나타냄. 가정이 옳바르다면 모델은 더 적은 데이터로 높은 일반화 성능을 보일 수 있지만, 너무 강한 귀납적 편향을 다른 유형의 데이터나 관계를 표현하는데 어려움을 초래할 수 있음

ViT 모델

- 입력 이미지를 트랜스포머 구조에 맞게 일정한 크기의 패치로 나눈 다음, 각 패치를 벡터 형태로 변환하는 **패치 임베딩(Patch Embedding)**과 각 패치와의 관계를 학습하는 **인코더(Encoder) 계층**으로 이미지 특징을 추출함.
- 이후 분류나 회귀와 같은 작업에 맞는 출력값으로 변환해 사용함패치 임베딩

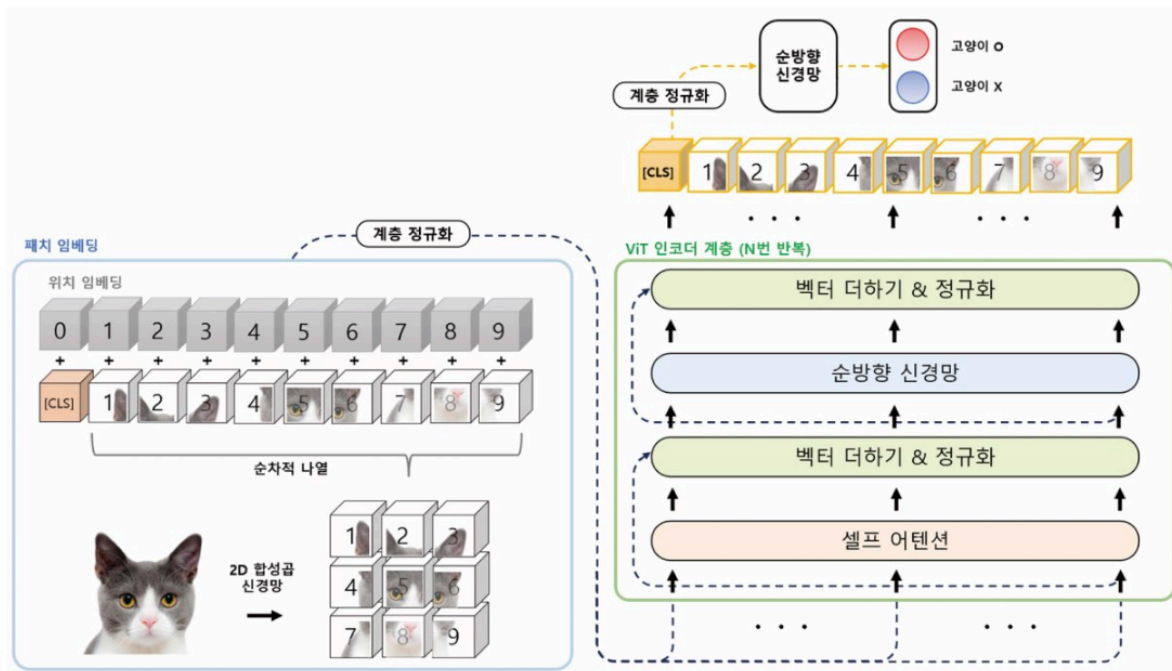


그림 10.4 ViT 모델 구조

패치 임베딩

- 입력 이미지를 작은 패치로 분할하는 과정
1. 작은 패치로 분할하기 위해 이미지 크기를 맞추는 전처리가 수행(664 x 480 → 224 x 224 정방향 크기)
 2. 전체 이미지를 패치 크기로 분할해 시퀀스 배열 생성 (합성곱 신경망 계층 활용)
 3. 하이퍼 파라미터 설정
 - 커널 크기(Kernel Size): 패치의 크기
 - 간격(Stride): 패치가 이동하는 폭

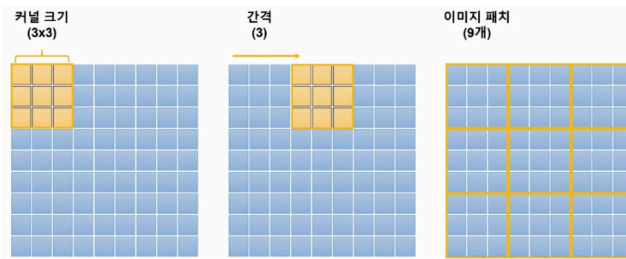


그림 10.5 이미지 패치 생성 과정

- 가로/세로 패치 개수 계산 식

$$patch\ size = \frac{image\ size - kernel\ size}{stride} = \frac{9 - 3}{3} + 1 = 3$$

4. 분할된 이미지 패치들을 왼 → 오, 위 → 아래 순서로 배열한다.
5. 가장 왼쪽에 **분류 토큰(Special Classification Token, CLS)**을 추가한다. 분류 토큰은 전체 이미지를 대표하는 벡터로 특정 문제를 예측하는데 사용된다.
6. **위치 임베딩(Position Embedding)**을 사용해 인접한 패치 간의 관계를 학습한다. 위치 임베딩은 패치의 위치 정보를 임베딩 벡터로 변환하고 기존 이미지 패치 벡터들과 더한다.
7. 계층 정규화를 적용하여 패치 임베딩을 완료한다. 이미지를 작은 패치로 나눠 처리하면 GPU 메모리의 한계를 극복하고 더 큰 이미지를 처리할 수 있다.

인코더 계층

- 패치 임베딩을 후, 인코딩 계층에 입력으로 전달해 다양한 쿼리, 키, 밸류 임베딩의 관계를 학습한다.
- N개의 인코더 레이어를 반복적으로 적용한 후, 마지막 레이어에서는 분류 토큰의 특징 벡터를 추출한다. 이 분류 토큰 벡터는 이미지 데이터를 잘 표현하는 특징 벡터로 간주된다. 다양한 이미지 분류 및 검색 문제를 해결하는데 사용된다.
 - 예를 들어, 고양이인지 아닌지 구분하는 모델을 학습한다면 ViT 모델은 **시퀀셜 산출물 (Sequential Output)**을 모두 사용하는 것이 아니라 전체 이미지 특징을 잘 표현하는 분류 토큰 벡터만 사용해서 분류 문제를 푼다.
 - 분류 토큰 벡터를 순방향 신경망 (or fc 계층)에 연결하고 고양이인지 아닌지를 분류하는 방법으로 ViT 모델이 학습된다.

실습

- 허깅 페이스 라이브러리와 FashionMNIST 데이터 세트를 활용해 ViT 모델을 미세 조정한다.

- Fashion MNIST 데이터세트는 의류 이미지를 담고 있는 데이터 세트로, 총 10개의 클래스와 60,000개의 훈련 데이터 세트, 10,000 개의 테스트 데이터 세트로 구성된다.

10.1 FashionMNIST Download

```
def subset_sampler(dataset, classes, max_len):
    target_idx = defaultdict(list)
    for idx, label in enumerate(dataset.targets):
        target_idx[int(label)].append(idx)

    indices = list(
        chain.from_iterable(
            [target_idx[idx][:max_len] for idx in range(len(classes))]
        )
    )
    return Subset(datasets, indices)

train_dataset = datasets.FashionMNIST(root='./datasets', download= True, tra
test_dataset = datasets.FashionMNIST(root='./datasets', download = True, tra

# classes 메서드로 데이터셋에 포함된 클래스를 확인할 수 있음
# class_to_idx로 클래스 ID와 클래스가 매핑된 값 확인할 수 있음
classes = train_dataset.classes
class_to_idx = train_dataset.class_to_idx

print(classes)
print(class_to_idx)

# subset_sanpler로 섭샘플링 (dataset, classes(클래스 목록), max_len(클래스별 최
subset_train_dataset = subset_sampler(
    dataset = train_dataset, classes = train_dataset.classes, max_len = 6000
)
subset_test_dataset = subset_sampler(
    dataset = test_dataset, classes = test_dataset.classes, max_len = 1000
)

print(f"Training Data Size : {len(subset_train_dataset)}")
```

```
print(f"Testing Data Size : {len(subset_test_dataset)}")
print(train_dataset[0])
```

- `defaultdict` 클래스를 활용해 클래스마다 6,000 개의 데이터 인덱스 값을 가져온다. (총 10개의 클래스)
 - 인덱스 값을 1차원 리스트로 풀어서 Subset 클래스에 전달하면 해당 인덱스 값에 해당하는 데이터셋을 추출할 수 있다.

10.2 이미지 전처리

- 모델을 학습하기 위해서는 `Tensor` 형태로 변환하는 전처리를 수행해야한다. 허깅 페이스의 `AutoImageProcessor` 를 사용해 전처리한다.

```
import torch
from torchvision import transforms
from transformers import AutoImageProcessor

image_processor = AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path = 'google/vit-base-patch16-224-in21k'
)

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Resize(
        size = (image_processor.size['height'], image_processor.size['width'])
    ),
    transforms.Lambda(lambda x: torch.cat([x, x, x], 0)),
    transforms.Normalize(mean = image_processor.image_mean,
        std = image_processor.image_std)
])

print(f"size : {image_processor.size}")
print(f"mean : {image_processor.image_mean}")
print(f"std : {image_processor.image_std}")
```

```
size : {'height': 224, 'width': 224}
mean : [0.5, 0.5, 0.5]
std : [0.5, 0.5, 0.5]
```

- **이미지 프로세서 클래스(AutoImageProcessor)**는 사전 학습된 ViT 모델을 활용해 전처리를 진행한다. 이미지 프로세서 클래스는 자르기나 크기 조정등의 전처리를 수행할 수 있다.
- 이 예제에서 사용한 `google/vit-base-patch16-224-in21k` 모델은 ImageNet-21k 데이터셋을 사용해 224 x 224 이미지 크기와 16 개의 이미지 패치를 기반으로 학습되었다.
- 모델 학습시 PIL.Image 형식이 아니라 `Tensor` 형식을 사용하므로 변환 모듈을 통해 이미지를 변환한다. 학습에 사용하려는 이미지를 ViT 모델과 맞추기 위해 `transforms.Resize`를 진행하며, 사용자 정의 함수 클래스 (`transforms.Lambda`) 단일 채널을 복제해 다중 채널 이미지로 변환한다. `transforms.Normalize` 를 적용해 정규화한다.
- ViT 모델은 Tensor 형식의 데이터를 사용하는 것이 아닌 딕셔너리 형식의 데이터를 입력으로 받는다.
 - **{"pixel_values": pixel_values, "Labels": Labels}** 의 형태를 입력으로 받음
 - 데이터로더 **집합 함수(collate_fn)**를 적용해 미니 배치의 샘플 목록을 병합한다. 이 함수로 ViT 모델 구조에 맞는 형태로 변환해 데이터로더에 적용한다.

10.3 ViT 데이터 로더 적용

```
from torch.utils.data import DataLoader

# ViT 모델에 맞는 입력 구조 변환 수행
def collator(data, transform):
    images, labels = zip(*data)
    pixel_values = torch.stack([transform(image) for image in images])
    labels = torch.tensor([label for label in labels])
    return {'pixel_values': pixel_values, 'labels': labels}
# dictionary 구조로 값 반환

train_dataloader = DataLoader(
    subset_train_dataset, batch_size = 32, shuffle = True,
    collate_fn = lambda x: collator(x, transform), drop_last = True
)
valid_dataloader = DataLoader(
```

```

subset_test_dataset, batch_size = 4, shuffle = True,
collate_fn = lambda x: collator(x, transform), drop_last = True
)

batch = next(iter(train_dataloader))
for key, value in batch.items():
    print(f'{key}: {value.shape}')

```

- 데이터 로더에 집합 함수를 적용하기 위해 사용자 정의 함수(`collator`)를 생성한다.
- `Collator` 함수에 `transform` 변수를 함께 전달해 이미지를 변환하고 `{"pixel_values": pixel_values, "labels": Labels}` 구조로 값을 반환한다.
 - `pixel_values` 는 (배치 크기 , 채널 수 , 이미지 높이 , 이미지 너비) 로 구성되며 `Labels` 는 클래스 index 값을 반환한다.
- `train_dataloader`의 첫 번째 값을 확인해 보면 딕셔너리 형식으로 데이터를 반환하고 배치 크기와 동일한 구조로 데이터가 생성된 것 확인 가능

10.4 사전 학습된 ViT 모델

```

from transformers import ViTForImageClassification

model = ViTForImageClassification.from_pretrained(
    pretrained_model_name_or_path = 'google/vit-base-patch16-224-in21k',
    num_labels = len(classes),
    id2label = {idx: label for label, idx in class_to_idx.items()},
    label2id = class_to_idx,
    ignore_mismatched_sizes = True
)

print(model.classifier)

```

```

model.safetensors: 100% ██████████ 346M/346M [00:05<00:00, 33.1ME
Some weights of ViTForImageClassification were not initialized from the n
You should probably TRAIN this model on a down-stream task to be able to
Linear(in_features=768, out_features=10, bias=True)

```

- 허깅 페이스 라이브러리의 **ViT 이미지 분류 모델 클래스 (ViTForImageClassification)**를 통해 사전 학습된 모델을 불러올 수 있다. 사전 학습 도니 모델은 이미지 프로세서 클래스에서 사용한 모델과 동일하다.
- 이 모델은 21,841 개의 분류 카테고리 학습된 모델이므로 미세 조정을 필요로 한다. 매개 변수를 통해 미세 조정 할 수 있다.
 - 레이블 개수 (`num_labels`), ID/ 레이블 (`id2label`), 레이블 /ID (`label2id`) 매개변수를 통해 현재 데이터 세트에 적합한 구조로 모델을 미세 조정한다
- 출력 결과를 확인해 보면 분류기 (`classifier`) 의 출력 데이터 차원 크기 (`out_features`) 를 10 개로 반환하는 것을 확인할 수 있다
- 모델 학습하기에 앞서 패치 임베딩 구조를 확인해본다.

10.5 패치 임베딩 확인

```
print(model.vit.embeddings)

batch = next(iter(train_dataloader))
print ("image shape :", batch["pixel_values"]. shape)
print("patch embeddings shape :",
      model.vit. embeddings.patch_embeddings(batch["pixel_values"]).shape)
print('[CLS] + patch.embeddings.shape:',
      model.vit.embeddings(batch['pixel_values']).shape)
```

```
ViTEmbeddings(
  (patch_embeddings): ViTPatchEmbeddings(
    (projection): Conv2d(3, 768, kernel_size=(16, 16), stride=(16, 16))
  )
  (dropout): Dropout(p=0.0, inplace=False)
)
image shape : torch.Size([32, 3, 224, 224])
patch embeddings shape : torch.Size([32, 196, 768])
[CLS] + patch.embeddings.shape: torch.Size([32, 197, 768])
```

- ViT model은 트랜스포머 모델과 유사하지만, 입력 데이터를 일정한 크기의 패치로 분할한 후 패치마다 특성을 추출하는 과정을 추가로 수행한다. 이를 위해 ViT model에는 **패치 임베딩 (`patch_embeddings`)**이 추가된다.
- 패치 임베딩의 처리 방식을 확인하기 위해 배치 데이터 하나를 **패치 임베딩 함수 (`model.vit.embeddings.patch_embeddings`)**에 적용해본다

- 패치 임베딩 함수는 16 x 16 커널과 16 stride의 합성곱(Conv2d) 연산을 수행한다. 224 x 224 크기의 이미지에 합성곱 연산을 수행하면 $(224-16)/16+1=14 \rightarrow 14 \times 14$ 의 텐서가 생성된다. 이 텐서를 일렬로 나열하면 196개의 패치가 생성된다.
- 패치 임베딩이 생성된 후 **임베딩 함수** (`model.vit.embeddings`)가 수행된다. 이 함수 앞에 [4, 1, 768] 크기의 [CLS] 토큰이 패치 앞에 붙어 197 개의 벡터가 생성된다.

10.6 하이퍼 파라미터 설정

- 트랜스포머스 라이브러리의 훈련자 클래스를 활용해 모델 학습

```
from transformers import TrainingArguments
args = TrainingArguments(
    output_dir = './models/ViT-FashionMNIST',
    save_strategy = 'epoch',
    eval_strategy = 'epoch',
    learning_rate = 1e-5,
    per_device_train_batch_size = 16,
    per_device_eval_batch_size = 16,
    num_train_epochs = 3,
    weight_decay = 0.01,
    load_best_model_at_end = True,
    metric_for_best_model = 'f1',
    logging_dir = 'logs',
    logging_steps = 125,
    remove_unused_columns = False,
    seed = 7
)
```

- **학습 매개변수 클래스(TrainingArguments)** 는 모델 학습에 필요한 다양한 인자들을 저장하고 관리한다. 하이퍼파라미터 관리, 모델 평가, 분산 학습, 허깅 페이스 허브 연동, 로그 추적 등의 기능을 제공하며, 80 여개의 매개 변수를 통해 다양한 기능을 설정할 수 있다. 이번 예제에서 사용된 매개 변수 및 주요한 매개 변수는 다음과 같다.

매개변수	의미
<code>output_dir</code>	체크포인트 저장 경로
<code>save_strategy</code>	체크포인트 저장 간격 설정 (<code>no</code> , <code>steps</code> , <code>epoch</code>)
<code>save_steps</code>	<code>save_strategy="steps"</code> 일 때 저장 간격 스텝 수
<code>save_total_limit</code>	체크포인트 최대 저장 개수

<code>evaluation_strategy</code>	평가 간격 설정 (<code>no</code> , <code>steps</code> , <code>epoch</code>)
<code>learning_rate</code>	초기 학습률
<code>per_device_train_batch_size</code>	학습 배치 크기
<code>per_device_eval_batch_size</code>	평가 배치 크기
<code>num_train_epochs</code>	학습 반복 수 (에폭 수)
<code>weight_decay</code>	가중치 감쇠
<code>load_best_model_at_end</code>	학습 종료 시 최상의 모델 불러오기 여부
<code>metric_for_best_model</code>	최상의 모델 선택 기준 평가 지표 (<code>accuracy</code> , <code>precision</code> , <code>f1</code> , <code>mae</code> , <code>mse</code> , <code>rmse</code>)
<code>logging_dir</code>	로그 저장 폴더
<code>logging_steps</code>	로그 출력 간격
<code>seed</code>	학습 시작 시 무작위 시드
<code>fp16</code>	Mixed precision 사용 여부 (<code>True</code> 면 FP16 사용)

10.7 매크로 평균 F1 점수

- 하이퍼 파라미터 설정이 완료되었다면, 모델의 성능을 평가하기 위한 성능 평가 지표 방법을 설정한다. 성능 평가 방법은 Macro Average F1-Score를 사용한다.
- 현재 예제는 다중 클래스이므로 모든 클래스에 대한 F1 점수의 가중 평균으로 전체 모델의 성능을 측정한다.
- 매크로 평균 F1 점수 평가 구현 방법을 실습한다.

```
import evaluate
import numpy as np

def compute_metrics(eval_pred):
    metric = evaluate.load('f1')
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis = 1)
    macro_f1 = metric.compute(
        predictions = predictions, references = labels, average = 'macro'
    )
    return macro, f1
```

- 성능 평가 지표도 허깅 페이스의 평가 라이브러리로 메트릭을 쉽게 불러올 수 있다.

- 메트릭 불러오기 함수 (load) 는 정확도 (accuracy), 정밀도 (precision), F1 점수 (F1) 등 다양한 평가 지표를 제공한다. 이 함수를 통해 F1 점수 방식을 불러온다.
- `eval_pred` 인수는 모델이 예측한 결괏값을 전달받는다. 모든 클래스에 대한 예측값이 포함돼 있으므로, 가장 확률이 높은 값을 선택해 `predictions` 변수에 저장한다.
- 예측 데이터를 정제했다면 예측값 (`predictions`)과 실제값 (`Labels`) 을 메트릭 연산 (`metric.compute`) 메서드에 전달한다. 현재 예제는 매크로 평균 F1 점수 방식을 사용하므로 평균 (average) 매개변수에 macro 를 입력한다.

10.8 ViT 모델 학습

```
def subset_sampler(dataset, classes, max_len):
    target_idx = defaultdict(list)
    for idx, label in enumerate(dataset.targets):
        target_idx[int(label)].append(idx)

    indices = list(
        chain.from_iterable(
            [target_idx[idx][:max_len] for idx in range(len(classes))]
        )
    )
    return Subset(dataset, indices)

def model_init(classes, class_to_idx):
    model = ViTForImageClassification.from_pretrained(
        pretrained_model_name_or_path="google/vit-base-patch16-224-in21k",
        num_labels= len(classes),
        id2label={idx: label for label, idx in class_to_idx.items()},
        label2id=class_to_idx,
    )
    return model

def collator(data, transform):
    images, labels = zip(*data)
    pixel_values = torch.stack([transform(image) for image in images])
    labels = torch.tensor([label for label in labels])
    return {'pixel_values': pixel_values, 'labels': labels} # dictionary 구조로 값 반환
```

```

def compute_metrics(eval_pred):
    metric = evaluate.load('f1')
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis = 1)
    macro_f1 = metric.compute(
        predictions = predictions, references = labels, average = 'macro'
    )
    return macro_f1

train_dataset = datasets.FashionMNIST(root='../datasets', download= True, tra
test_dataset = datasets.FashionMNIST(root='../datasets', download = True, tra

classes = train_dataset.classes
class_to_idx = train_dataset.class_to_idx

print(classes)
print(class_to_idx)

subset_train_dataset = subset_sampler(
    dataset = train_dataset, classes = train_dataset.classes, max_len = 1000
)
subset_test_dataset = subset_sampler(
    dataset = test_dataset, classes = test_dataset.classes, max_len = 100
)

image_processor = AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path = 'google/vit-base-patch16-224-in21k'
)

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Resize(
        size = (image_processor.size['height'], image_processor.size['width'])
    ),
    transforms.Lambda(lambda x: torch.cat([x, x, x], 0)),
    transforms.Normalize(mean = image_processor.image_mean,
        std = image_processor.image_std)
])

```

```
])
```

```
args = TrainingArguments(
    output_dir = './models/ViT-FashionMNIST',
    save_strategy = 'epoch',
    eval_strategy = 'epoch',
    learning_rate = 1e-5,
    per_device_train_batch_size = 16,
    per_device_eval_batch_size = 16,
    num_train_epochs = 3,
    weight_decay = 0.01,
    load_best_model_at_end = True,
    metric_for_best_model = 'f1',
    logging_dir = 'logs',
    logging_steps = 125,
    remove_unused_columns = False,
    seed = 7
)

trainer = Trainer(
    model_init = lambda x: model_init(classes, class_to_idx),
    args = args,
    train_dataset = subset_train_dataset,
    eval_dataset = subset_test_dataset,
    data_collator = lambda x: collator(x, transform),
    compute_metrics = compute_metrics,
    tokenizer = image_processor,
)
```

- 트랜스포머 라이브러리의 **훈련자 클래스(`Trainer`)**를 통해 ViT 모델의 학습을 수행한다. 훈련자 클래스는 모델을 함수(`model_init`)의 형태로 생성해 전달하고, 데이터로더가 내부적으로 수행되므로 데이터셋과 집합 함수 등을 함께 전달한다.
- 토크나이저(`tokenizer`)는 `image_processor` 를 전달한다. 토크나이저는 토큰을 임베딩해 모델 입력으로 사용하도록 토큰 분리 작업을 수행한다.
- 학습이 완료되면 ViT-FashionMNIST 폴더에 체크 포인트마다 학습 결과, 성능 지표, 하이퍼 파라미터 등이 저장된다.

Epoch	Training Loss	Validation Loss	F1-Score
1	0.7062	0.6377	0.8905
2	0.4954	0.4781	0.9166
3	0.4078	0.4357	0.9231

- 최종 학습 결과의 손실값이 0.4078이며, F1 점수가 0.9231로 학습이 우수하게 진행된 것을 확인할 수 있다.

10.9 ViT 모델 성능 평가

```
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

output = trainer.predict(subset_test_dataset)
print(output)

y_true = output.label_ids
y_pred = output.predictions.argmax(1)

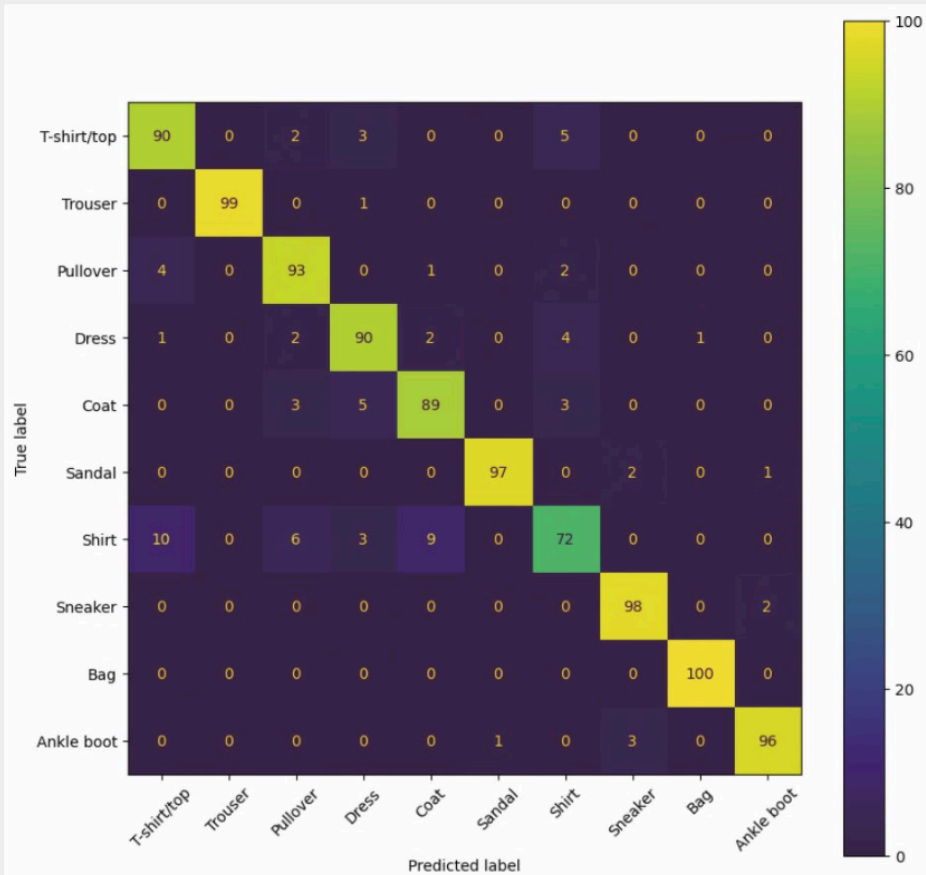
labels = list(classes)
matrix = confusion_matrix(y_true, y_pred)
display = ConfusionMatrixDisplay(confusion_matrix = matrix, display_labels =
_, ax = plt.subplots(figsize = (10, 10))
display.plot(xticks_rotation = 45, ax = ax)
plt.show()
```

```
# 생략
{
    'test_loss': 0.43576720356941223,
```

```

'test_f1': 0.9231955290222554,
'test_runtime': 8.4356,
'test_samples_per_second': 29.636,
'test_steps_per_second': 4.283
}
# 생략

```



- 테스트 데이터셋으로 ViT 모델을 테스트하면 약 92%의 분류 정확도를 갖는다. (준수한 성능)
- 하지만 confusion matrix로 클래스별 성능 지표를 확인해보면 셔츠를 티셔츠나 코트로 오분류한 경우가 상대적으로 많은 것을 확인할 수 있다.
- confusion matrix로 오분류된 클래스에 대한 문제점을 파악하고 모델을 개선해 정확도를 향상시킬 수 있다. 주요 모델 개선 방법으로는 하이퍼 파라미터 조정, 모델 구조 변경, 전처리 과정 개선, 데이터 증강 등이 있다.

Swin Transformer

- **Swin Transformer**는 2021년에 발표된 대규모 비전 인식 모델이다.

- 기존 트랜스포머를 기반으로 하는 모델의 공통적인 문제점은 고정된 패치 크기로 인해 세밀한 예측을 필요로 하는 *의미론적 분할과 같은 작업에 적합하지 않다*는 점이었다.
- 트랜스포머의 셀프 어텐션은 입력 이미지 크기에 대한 2차 계산 복잡도를 가지므로 고 해상도 이미지를 처리하기가 어려웠다. 이런 문제를 극복하기 위해 **계층적 특징 맵을 구성해 1차 계산 복잡도를 갖는 스윈 트랜스포머**가 등장했다.
- 스윈 트랜스포머는 이미지 분류, 객체 감지, 영상 분할과 같은 인식 작업에서 강력한 성능을 내며, 트랜스포머 구조보다 이미지 특성을 더 잘 표현할 수 있다.
- 스윈 트랜스포머는 **시프티드 윈도우(shifted window)**라는 기술을 이용해 입력 이미지를 일정한 크기의 패치로 분할하고 이 패치들을 쌓아 윈도를 구성한다. 이렇게 구성된 영역만 **셀프 어텐션을 계산한다**.
 - 시프티드 윈도 방식은 기존 트랜스포머 모델과 다르게 **패치를 겹쳐** 더 큰 효율성을 제공한다. 또한 계층마다 어텐션을 수행되는 패치의 크기와 개수를 계층적으로 적용해 처리하므로 **높은 해상도와 다양한 크기를 가진 객체를 효율적으로 처리할 수 있다**.
- 객체 탐지나 객체 분할과 같은 작업에서 백본 모델로 사용된다.

ViT와 스윈 트랜스 포머 차이

ViT

- 획일적인 패치 크기와 위치에 대한 제약 때문에 다양한 이미지 크기와 종횡비를 가진 데이터 세트에 대해서는 적용이 어렵다.
- 패치 단위로 입력을 받기 때문에 이미지의 공간 정보가 완전히 보존되지 않을 수 있다.
- 이미지의 패치 수가 증가하면 학습 데이터의 크기도 커지기 때문에 모델 학습에 필요한 계산량이 증가한다. 이로 인해 대규모 데이터셋에서는 학습에 필요한 컴퓨팅 자원을 많이 사용하게 되는 단점이 있다.

스윈 트랜스포머

- 이러한 한계를 극복하기 위해 **로컬 윈도 (Local Window)**를 활용해 물체의 크기나 해상도를 **계층적으로 학습**한다. 로컬 윈도는 스윈 트랜스포머에서 입력 이미지를 처리하기 위해 사용되는 고정 크기의 작은 윈도다.
- 이러한 로컬 윈도를 이동시키는 시프트 윈도 기술을 도입해 입력 이미지를 일정한 크기의 패치로 분할, 특징 맵을 이동시켜 다음 계층에 사용함으로써 패치의 위치를 더 유연하게 다룬다. 이를 통해 스윈 트랜스포머는 ViT보다 **더 높은 정확도와 더 적은 매개 변수 수를 가지면서도 더 넓은 범위의 이미지 크기와 종횡비를 다룰 수 있다**.

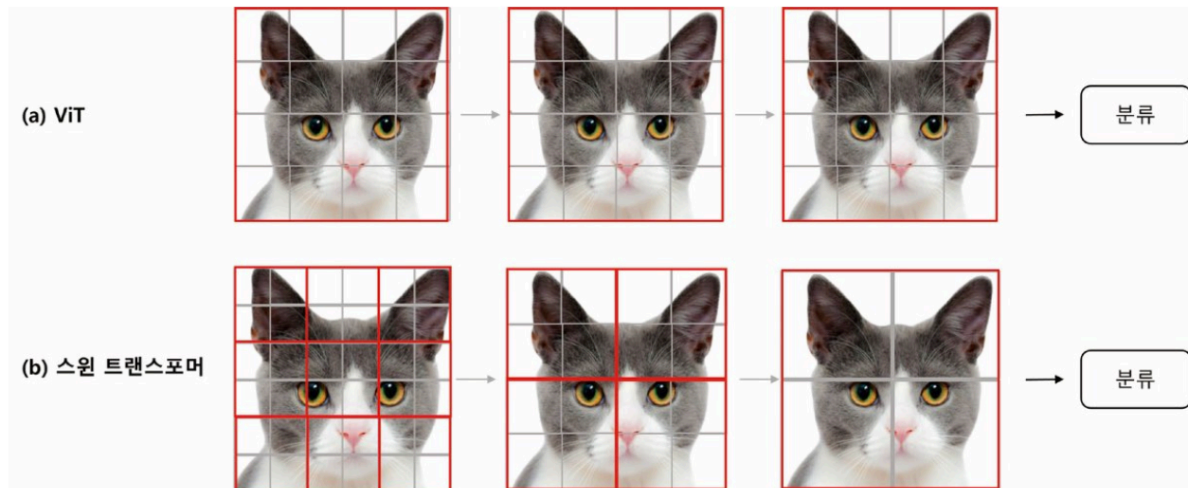


그림 10.6 ViT와 스윈 트랜스포머의 네트워크 구조

- 회색 격자: 패치, 빨간색 격자: 로컬 윈도우
- ViT 의 네트워크는 계층마다 전체 이미지 안에 어텐션이 수행되는 패치 크기와 개수가 동일하지만
- 스윈 트랜스포머는 로컬 윈도우를 통해 계층마다 어텐션이 되는 패치의 크기와 개수를 계층적으로 다양하게 적용한다.
 - 즉, 입력 패치의 로컬 윈도우를 통해 계층적으로 접근하고 시프트 윈도우로 로컬 윈도우를 이동해 가면서 인접 패치 간 정보를 계산한다
- 이러한 계층적 접근 방식은 로컬 윈도우가 각 패치의 세부 정보에 집중하게 되고, 시프트 윈도우를 통해 전역 특징을 확인할 수 있으므로 **고해상도 이미지를 효율적으로 처리할 수 있다.**

	ViT	스윈 트랜스포머
분류 헤드	[CLS] 토큰 사용	각 토큰의 평균값 사용
로컬 윈도우 사용	X	O
상대적 위치 편향 사용	X	O
계층별 패치 크기	동일함	다양함

스윈 트랜스포머 모델 구조

- 이미지를 패치 파티션으로 구분한 후, 선형 임베딩(Linear Embedding), 스윈 트랜스포머 블록(Swin Transformer Block), 패치 병합 연산(Patch Merging)이 반복적으로 수행된다.

1. 입력 이미지를 일정한 크기의 배치로 분할
2. 각 패치에 선형 임베딩 수행 → 각 패치는 고정된 차원의 벡터로 변환
3. 분할된 패치들을 기반으로 스윈 트랜스포머 블록을 구성한다. (어텐션 계층, 계층 정규화, 다층 퍼셉트론을 포함하며 패치간 상호작용 수행)
4. 패치를 병합해 전체 이미지에 대한 분류 수행
5. 분할된 패치들을 순차적으로 합치면서 이미지의 전체적인 정보를 추출하는 방식을 사용

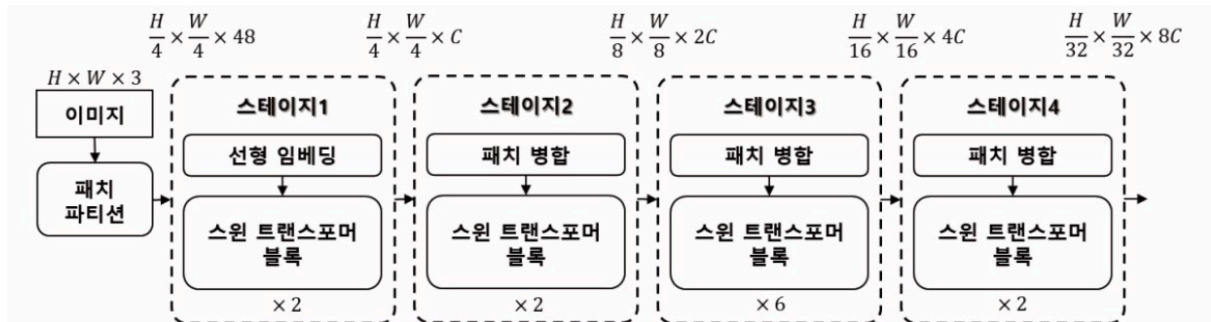
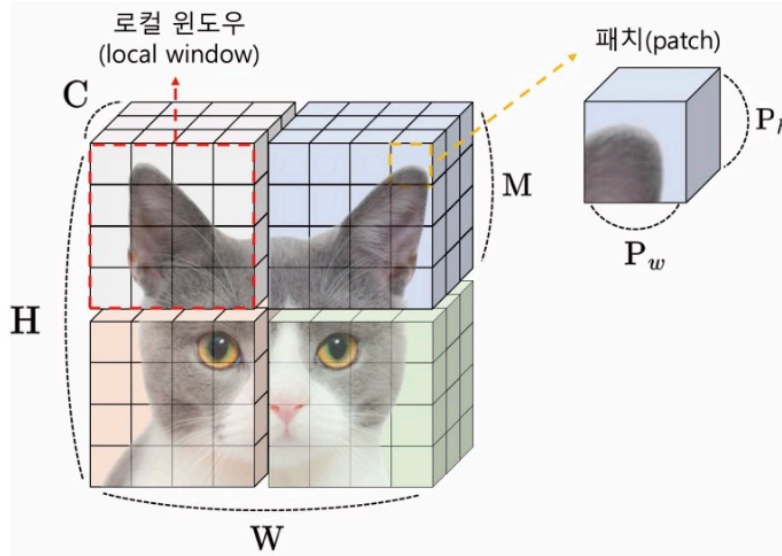


그림 10.7 스윈 트랜스포머 구조

- 일반적으로 4 개의 스테이지로 구성되어 있으며, 각 스테이지는 서로 다른 해상도와 패치 크기를 가짐
 - 선형 임베딩 스윈 트랜스포머 블록, 패치 병합이 반복해 적용된다.
- 스테이지1에서는 패치들이 선형 임베딩된 후 스윈 트랜스 포머 블록에 전달
- 스테이지 2, 3, 4에서는 패치 병합후 각 스윈 트랜스포머 블록에 전달됨. 패치 병합은 모델의 매개 변수를 줄이기 위해 이미지 텐서 $[C, H, W]$ 를 $[2C, H/2, W/2]$ 로 재정렬해 저차원 임베딩을 수행
- 스테이지 1, 2, 3, 4 에서 공통적으로 사용되는 스윈 트랜스포머 블록은 윈도우 격자 안에 있는 패치 내부의 관계 , 윈도우 격자 간의 관계를 학습하는 두 가지 어텐션 모듈로 이뤄져 있다.

패치 파티션/패치 병합

- 패치 파티션
 - 입력 이미지를 작은 사각형 패치로 분할해 처리하는 방식
 - 분할된 패치는 트랜스포머 계층의 입력으로 사용되며, 입력 이미지의 공간 정보를 보존하고 트랜스포머 계층에 대한 계산 효율성을 높인다.



- 입력 이미지 텐서는 $[C, H, W]$ 의 구조를 갖는다. 이 구조에서 패치 파티션을 통해 로컬 윈도우를 추가한다. (M : 로컬 윈도우 크기)
- 이미지 텐서의 차원이 $[3, 32, 32]$ 이라면 패치의 가로 (P) 와 세로 (P) 는 4 가 된다.

• 패치 병합

- 인접한 패치들의 정보를 저차원으로 축소하는 과정
- 모델 매개변수의 수를 줄이기 위해 이미지 패치 텐서 $[C, H, W]$ 를 $[2C, H/2, W/2]$ 로 재정렬한 후, $2C$ 의 차원을 저차원으로 임베딩
 - Ex. 채널 (C) 3 개 , 이미지 패치 ($H \times W$) 64 개로 구성된 텐서 $[3, 8, 8]$. 정의된 $[2C, H/2, W/2]$ 에 따라 $[6, 4, 4]$ 로 재정렬한다. 증가된 6 차원 채널을 원래 3 차원으로 임베딩해 산출물 $[3, 4, 4]$ 텐서 생성

스윈 트랜스포머 블록

- 패치 파티션 이후 네 개의 스테이지 안에서 반복 적용되며, 선형 임베딩 또는 패치 병합 이후에 수행된다.
- 스윈 트랜스포머 블록은 계층 정규화 , W-MSA(Window Multi-head Self-Attention), MLP, SW-MSA(Shifted Window Multi-head Self-Attention) 등으로 구성된다.

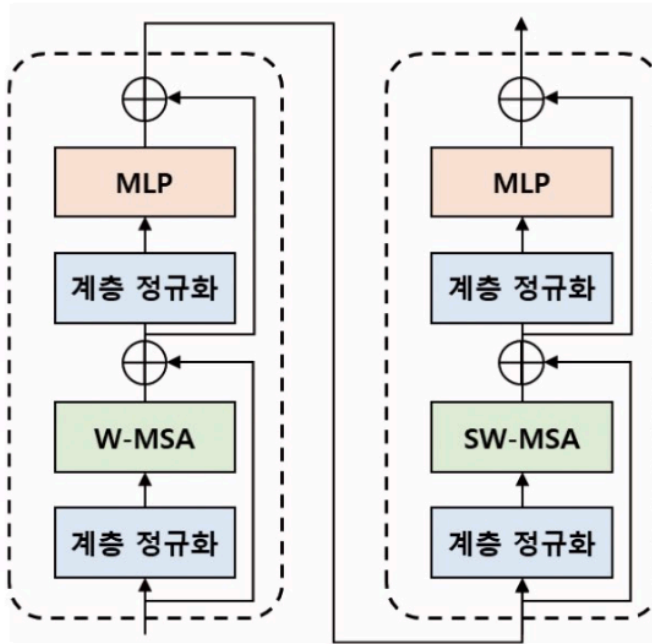


그림 10.9 스윈 트랜스포머 블록 구조

- 스윈 트랜스포머 블록 안에서는 트랜스포머 모델에서 사용되던 MSA(Multi-head Self-Attention) 방식이 아닌 **W-MSA** 와 **SW-MSA** 로 구성되며 순차적으로 수행된다.

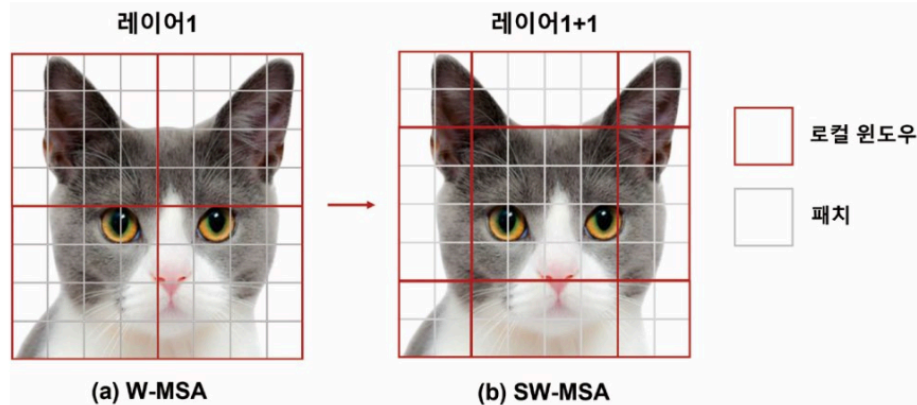


그림 10.10 W-MSA와 SW-MSA 어텐션 영역 비교

- **W-MSA**: 로컬 윈도우를 사용하는 멀티 헤드 셀프 어텐션 (입력 특징 맵을 중첩되지 않게 나눈 다음 각 윈도우에서 **독립적으로** 셀프 어텐션을 수행).
 - 이미지에서 일정한 크기의 윈도우 영역을 설정하고, 해당 영역 내 픽셀 간의 어텐션을 계산한다.
 - 이를 통해 이미지 내의 전체적인 어텐션을 수행하고, 특정 위치에 대한 정보를 추출한다.

- 각 윈도우 내 지역적인 특징을 분석할 수 있으며, 전체 입력에 대한 셀프 어텐션 비용을 2 차 계산 복잡도에서 1차 계산 복잡도로 줄인다.
- 하지만 윈도우 내부의 이미지 패치 영역에서만 셀프 어텐션을 수행한다는 단점이 있다.
- 로컬 윈도우 (빨간색 영역) 안에서 패치 간의 셀프 어텐션 연산을 수행한다. 로컬 윈도우가 이동하면서 각각의 윈도우에서 연산을 수행해야하므로 계산량이 많아지는 것처럼 보일 수 있지만, **효율적인 배치 계산을 통해 연산량을 감소시킴**

○ SW-MSA

- W-MSA의 단점을 보완하기 위해 등장.
- 윈도우 사이의 연결성 구축해 윈도우 간의 관계성 정보를 추출한다. SW-MSA 는 윈도우를 가로 또는 세로 방향으로 이동한 후, 인접한 윈도우 간의 셀프 어텐션을 계산하는 방식
- 로컬 윈도우 간의 셀프 어텐션을 수행하기 위해 W-MSA에서 사용된 로컬 윈도우 개수보다 많아 더 많은 연산을 요구한다. 이를 해결하기 위해 **순환 시프트와 어텐션 마스크**를 사용했다.

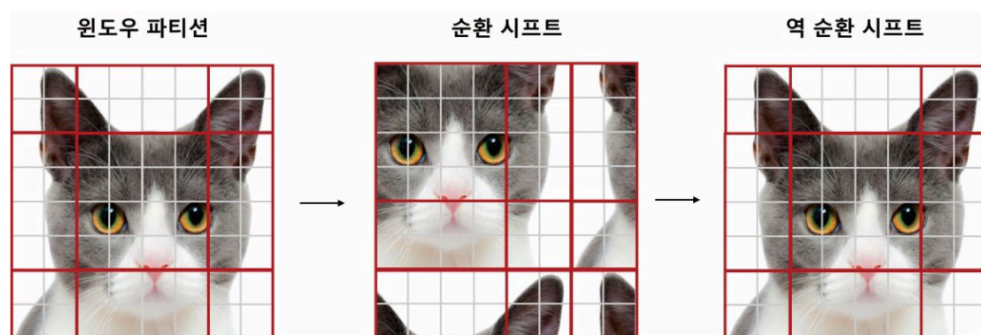


그림 10.11 순환 시프트를 활용한 SW-MSA 어텐션 계산 과정

- **순환 시프트**: 로컬 윈도우가 이동할 때 이동한 위치 정보를 이전 위치에서 가져오는 것, $M/2$ 만큼 로컬 윈도우들을 이동시킴
- 이동된 위치에서 연산을 수행하지 않도록 **어텐션 마스크**를 사용해 방지.
 - 원래 총 9 개의 윈도우 파티션에 대해 9 번의 셀프 어텐션 값을 구해야 했지만, 어텐션 마스크를 사용하면 4 개의 윈도우 파티션에 대한 4 개의 셀프 어텐션 값만으로 모든 로컬 윈도우 간의 셀프 어텐션 값을 계산할 수 있게 된다.
- **역순환 시프트**: 순환 시프트 단계에서 셀프 어텐션을 수행한 다음 사용하여 원래 로컬 윈도우 구조로 복원함

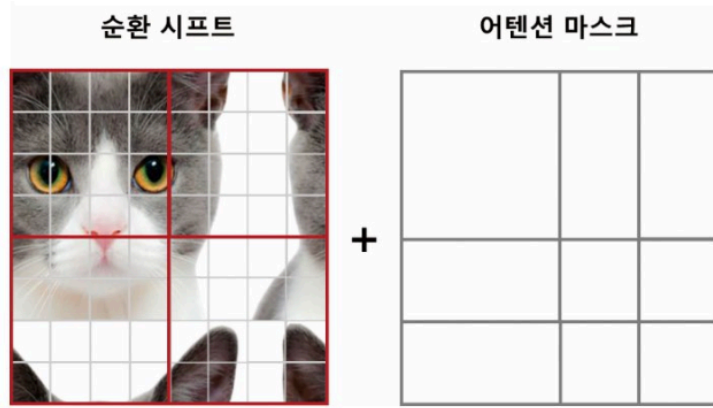


그림 10.12 순환 시프트 단계에서 마스크를 활용한 셀프 어텐션 구조

- 기존 ViT와 달리 스윈 트랜스포머에서 사용한 셀프 어텐션은 **상대적 위치 편향을 고려**한다. 상대적 위치 편향은 로컬 윈도우 안에 있는 패치 간의 상대적인 거리를 임베딩하는 목적을 가진다.



그림 10.13 상대적 위치 편향 계산 과정

- 이미지 패치들이 1부터 4까지 주어진다면 패치 간의 상대적인 거리는 X축과 Y축 두 가지 방식으로 표기할 수 있다.
 - X축 방식: 두 패치 간의 가로 방향 거리,
 - X 축에 놓여 있으면 상대적 거리가 0
 - 아래로 떨어져 있다면 1
 - 위로 떨어져 있으면 1
 - Y축 방식: 세로 방향 거리

- 같은 Y 축에 놓여 있으면 상대적 거리가 0
- 오른쪽으로 떨어져 있으면 -1
- 왼쪽으로 떨어져 있으면 1

10.10 상대적 위치 편향 계산

```
import torch

window_size = 2
# 여러 개의 이미지 패치를 포함하는 격자 크기
# 윈도우 내외부의 패치를 구분

coords_h = torch.arange(window_size)
coords_w = torch.arange(window_size)

# torch.meshgrid: coords_h 배열 값(i)과 coords_w 배열 값(j) 으로 사각형 격자 (1)
# torch.stack: X, Y 쌍 배열을 만듦
coords = torch.stack(torch.meshgrid([coords_h, coords_w], indexing = 'ij'))

# torch.flatten: 각 X, Y 축에 대한 위치 인덱스 생성
coords_flatten = torch.flatten(coords, 1)
relative_coords = coords_flatten[:, :, None] - coords_flatten[:, None, :]
print(relative_coords)
print(relative_coords.shape)
```

```
tensor([[[[ 0,  0, -1, -1],
          [ 0,  0, -1, -1],
          [ 1,  1,  0,  0],
          [ 1,  1,  0,  0]],
        [[ 0, -1,  0, -1],
          [ 1,  0,  1,  0],
          [ 0, -1,  0, -1],
          [ 1,  0,  1,  0]]]])
torch.Size([2, 4, 4])
```

10.11 X, Y 축에 대한 위치 행렬

```
x_coords = relative_coords[0, :, :]
y_coords = relative_coords[1, :, :]
```

```

x_coords += window_size - 1
y_coords += window_size - 1
x_coords *= 2 * window_size - 1
print(f"X축에 대한 행렬 :\n{x_coords}\n")
print(f"Y축에 대한 행렬:\n{y_coords}\n")

relative_position_index = x_coords + y_coords
print(f'X, Y 축에 대한 위치 행렬:\n{relative_position_index}')

```

X축에 대한 행렬 :

```

tensor([[3, 3, 0, 0],
        [3, 3, 0, 0],
        [6, 6, 3, 3],
        [6, 6, 3, 3]])

```

Y축에 대한 행렬:

```

tensor([[1, 0, 1, 0],
        [2, 1, 2, 1],
        [1, 0, 1, 0],
        [2, 1, 2, 1]])

```

X, Y 축에 대한 위치 행렬:

```

tensor([[4, 3, 1, 0],
        [5, 4, 2, 1],
        [7, 6, 4, 3],
        [8, 7, 5, 4]])

```

- `relative_coords` 를 각 X, Y 좌표에 대한 `x_coords, y_coords` 로 분리한 후, 각각 `window_size - 1` 만큼 더해준다 (③번 연산 과정).
- `x_coords` 에 대해서 `2 * window_size - 1` 만큼 곱해준다 (④번 연산 과정).
- 마지막으로 `x_coords` 와 `y_coords` 를 더해주면 X, Y 축에 대한 상대적 위치 좌표 행렬을 생성할 수 있다 (⑤번 연산 과정).

10.12 X, Y 축에 대한 상대적 위치 좌표 변환

```

num_heads = 1 # MSA 계산할 때 사용된 헤드 수
relative_position_bias_table = torch.Tensor(
    torch.zeros ((2 * window_size - 1) * (2 * window_size - 1), num_heads))

```

```
relative_position_bias = relative_position_bias_table[relative_position_index.view(
    relative_position_bias = relative_position_bias.view(
        window_size * window_size, window_size * window_size, -1
    )
    print(relative_position_bias.shape)
```

`torch.Size([4, 4, 1])`

- `relative_position_bias_table` 은 헤드별로 0~8 번까지 9 개의 상대적 위치 편향을 가지는 텐서를 의미한다.
- `relative_position_index` 를 이용해 해당 위치의 편향 값을 불러온다. 이 값들은 `view` 함수를 이용해 헤드별 i 번째 패치에서 i 번째 패치의 어텐션 값에 더해지는 텐서 `[4, 4, 1]` 로 재구성된다.
- 이렇게 생성된 상대적 위치 임베딩 (B) 을 기존 트랜스포머에 사용되는 셀프 어텐션 수식에 적용하면 다음 식과 같다.

수식 10.2 상대적 위치 편향을 고려한 셀프 어텐션

$$Attention(Q, K, V) = SoftMax(QK^T / \sqrt{d} + B)V$$

- Q, K, V: 쿼리, 키, 밸류 벡터
- d: 벡터들의 임베딩 차원

실습

- 앞선 ViT 모델의 실습 코드를 활용해 스윈 트랜스포머 모델을 미세 조정해 본다.
- 허깅 페이스 라이브러리에서 제공하는 사전 학습된 `microsoft/swin-tiny-patch4-window7-224` 모델을 사용
 - ImageNet-1k 데이터셋을 사용해 224X 224의 이미지 크기. 4X4 의 이미지 패치. 7X7 의 로컬 윈도우로 학습됐다

```
from transformers import SwinForImageClassification
```

```
model = SwinForImageClassification.from_pretrained(
    pretrained_model_name_or_path = 'microsoft/swin-tiny-patch4-window7-224'
```



```

num_labels = len(train_dataset.classes),
id2label = {idx: label for label, idx in train_dataset.class_to_idx.items()},
label2id = train_dataset.class_to_idx,
ignore_mismatched_sizes = True
)

for main_name, main_module in model.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print("L", sub_name)
        for ssub_name, ssub in sub_module.named_children():
            print("| L", ssub_name)
            for sssub_name, sssub_module in ssub_module.named_children():
                if ssub_name == 'projection':
                    print("| | L", sssub_name, sssub_module)
                else:
                    print("| | L", sssub_name)

```

```

swin
├─ embeddings
│   └─ patch_embeddings
│       └─ projection Conv2d(3, 96, kernel_size=(4, 4), stride=(4, 4))
│   └─ norm
│   └─ dropout
├─ encoder
│   └─ layers
│       └─ 0
│       └─ 1
│       └─ 2
│       └─ 3
├─ layernorm
├─ pooler
└─ classifier

```

- 스윈 트랜스포머는 크게 `swin` 과 `classifier` 로 구분된다. `swin` 에서는 패치 파티션, 선형 임베딩과 함께 네 개의 스테이지 (0, 1, 2, 3)가 수행된다. 스테이지 이후 계층 정규화 (`layernorm`)와 풀링(`pooler`)를 통과한다. `classifier` 는 클래스를 구분한다.
- 패치 파티션과 선형 임베딩은 `model.swin.embeddings.patch_embeddings` 에서 수행된다. `patch_embeddings` 에서 수행되는 합성곱 계층은 4x4 커널 크기, 4 간격으로 적용되었음.

이 계층에 224 x 224 이미지를 적용하면 56 x 56 텐서가 생성된다. 이 텐서를 일렬로 나열하면 3136 개의 패치가 생성된다.

10.14 패치 임베딩 모듈

```
batch = next(iter(train_dataloader))
print("이미지 차원:", batch['pixel_values'].shape)

patch_emb_output, shape = model.swin.embeddings.patch_embeddings(batch)
print("모듈:", model.swin.embeddings.patch_embeddings)
print("패치 임베딩 차원:", patch_emb_output.shape)
```

```
이미지 차원 : torch.Size([32, 3, 224, 224])
모듈: SwinPatchEmbeddings(
  (projection): Conv2d(3, 96, kernel_size=(4, 4), stride=(4, 4))
)
패치 임베딩 차원 : torch.Size([32, 3136, 96])
```

- 스윈 트랜스포머 모델의 패치 임베딩을 수행하면 3,136 개의 패치가 생성되며 96 개의 채널이 생성된다.
- ViT 모델은 [CLS] 토큰이 패치 앞에 붙어 197 개의 패치가 생성됨
- 하지만 스윈 트랜스포머는 [CLS] 토큰 사용 x, 토큰들의 **평균값**을 사용해 분류를 진행
- 스윈 트랜스포머 모델은 객체 검출과 같은 작업에서 입력 이미지에 여러 물체가 포함되면 [CLS] 토큰 하나로는 모든 물체를 효과적으로 검출하기 어렵기 때문에 평균값을 채택해 모델의 성능을 향상 시킨다.

10.15 스윈 트랜스포머 블록

```
for main_name, main_module in model.swin.encoder.layers[0].named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print('L', sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("| L", ssub_name)
```

```

blocks
├ 0
│   ├── layernorm_before
│   ├── attention
│   └── drop_path

```

```

│   ├── layernorm_after
│   ├── intermediate
│   └── output
├ 1
│   ├── layernorm_before
│   ├── attention
│   ├── drop_path
│   ├── layernorm_after
│   ├── intermediate
│   └── output
downsample
├ reduction
└ norm

```

- 스윈 트랜스포머의 스테이지는 **blocks** 와 **downsample** 로 이루어져있다.
 - blocks** : 스윈 트랜스포머 블록
 - downsample** : 패치 병합
-
- 스윈 트랜스포머 블록은 다시 **SwinLayer-0** 과 **SwinLayer-1** 로 이뤄져있다.
 - SwinLayer-0** : W-MSA
 - SwinLayer-1** : SW-MSA

10.16 SwinLayer 구조

```
print(model.swin.encoder.layers[0].blocks[0])
```

```

SwinLayer(
  (layernorm_before): LayerNorm((96,), eps=1e-05, elementwise_affine=True)
  (attention): SwinAttention(
    (self): SwinSelfAttention(
      (query): Linear(in_features=96, out_features=96, bias=True)
      (key): Linear(in_features=96, out_features=96, bias=True)
      (value): Linear(in_features=96, out_features=96, bias=True)
      (dropout): Dropout(p=0.0, inplace=False)
    )
    (output): SwinSelfOutput(
      (dense): Linear(in_features=96, out_features=96, bias=True)
    )
  )
)

```

```

        (dropout): Dropout(p=0.0, inplace=False)
    )
)
(drop_path): SwinDropPath(p=0.1)
(layer_norm_after): LayerNorm((96,), eps=1e-05, elementwise_affine=True)
(intermediate): SwinIntermediate(
  (dense): Linear(in_features=96, out_features=384, bias=True)
  (intermediate_act_fn): GELUActivation()
)
(output): SwinOutput(
  (dense): Linear(in_features=384, out_features=96, bias=True)
  (dropout): Dropout(p=0.0, inplace=False)
)
)

```

- 스윈 트랜스포머의 **SwinLayer** 는 기본적인 빌딩 블록이 되는 계층으로, 스윈 트랜스포머 블록을 의미한다.
- 스윈 트랜스포머 블록은 계층 정규화 → W-MSA (or SW-MSA) → 계층 정규화 → MLP 순서로 수행됨
- layernorm_before** 은 첫 번째 계층 정규화, **attention** 은 W-MSA(SW-MSA), **layernorm_after** 은 두 번째 계층 정규화, **intermediate, output** 은 MLP를 의미한다.
- MLP 계층(**intermediate, output**)을 살펴보면 두 개의 연속된 선형 변환 계층과 **GELU (Gaussian Error Linear Unit)** 활성화 함수로 구성된다.
 - GELU는 ReLU와 유사한 형태를 가지며, **가우시안 분포에 대한 누적함수**이다. GELU는 입력값 0 주변에서 부드럽게 변화한다.
- 스윈 트랜스포머는 두 개의 연속된 선형 변환과 GELU 함수를 통해 입력 특징맵을 더 깊은 차원으로 매핑하고, ReLU를 사용했을 때보다 더 안정적으로 학습이 이뤄진다.

10.17 W-MSA, SW-MSA 모듈

```

print("패치 임베딩 차원 :", patch_emb_output.shape)

W_MSA = model.swin.encoder.layers[0].blocks[0]
SW_MSA = model.swin.encoder.layers[0].blocks[1]

W_MSA_output = W_MSA(patch_emb_output, W_MSA.input_resolution)[0]
SW_MSA_output = SW_MSA(W_MSA_output, SW_MSA.input_resolution)[0]

```

```
print("W-MSA 결과 차원 :", W_MSA_output.shape)
print("SM-MSA 결과 차원 :", SW_MSA_output.shape)
```

```
패치 임베딩 차원 : torch.Size([32, 3136, 96])
W-MSA 결과 차원 : torch.Size([32, 3136, 96])
SW-MSA 결과 차원 : torch.Size([32, 3136, 96])
```

- 입력 텐서로 사용한 패치 임베딩 차원, W-MSA 모듈, SW-MSA 모듈의 출력 차원이 동일한 하이퍼 파라미터로 구성되었다.

10.18 패치 병합

```
patch_merge = model.swin.layers[0].downsample
print("patch_merge 모듈:", patch_merge)

output = patch_merge(SW_MSA_output, patch_merge.input_resolution)
print("patch_merge 결과 차원:", output.shape)
```

```
patch_merge 모듈 : SwinPatchMerging(
  (reduction): Linear(in_features=384, out_features=192, bias=False)
  (norm): LayerNorm((384,), eps=1e-05, elementwise_affine=True)
)
patch_merge 결과 차원 : torch.Size([32, 784, 192])
```

- 패치 병합은 입력 배치를 절반 크기로 분할하는 **reduction** 과 계층 정규화를 수행한다. 3,134 개의 패치 (56 x 56) → 784 (28 x 28) 개의 패치
- 분할된 패치는 4 개로 나뉘어 (N, 28, 28, 96)의 텐서가 생성된다. 이 네 개의 텐서를 채널에 병합하면 $96 \times 4 = 384$ 가 된다. 이렇게 하면 채널이 줄어들지 않으므로 선형 변환 계층을 통해 채널의 수를 절반으로 조정한다. (384 → 192)
- **patch_merge** 의 결과 차원은 [N, 784, 192]가 된다. 이와 같은 방식으로 총 네 개의 스테이지를 진행한다. 스테이지가 지날수록 패치 크기와 수는 많아지며 토큰의 차원은 두 배씩 증가한다. 마지막 스테이지를 통과하면 [N, 49(7×7), 768]이 된다. 이 특징맵이 풀링 계층을 통과해 클래스를 예측한다.

10.19 스윈 트랜스포머 모델 학습

```
from transformers import SwinForImageClassification
```

```

def subset_sampler(dataset, classes, max_len):
    target_idx = defaultdict(list)
    for idx, label in enumerate(dataset.targets):
        target_idx[int(label)].append(idx)

    indices = list(
        chain.from_iterable(
            [target_idx[idx][:max_len] for idx in range(len(classes))]
        )
    )
    return Subset(dataset, indices)

def model_init(classes, class_to_idx):
    model = SwinForImageClassification.from_pretrained(
        pretrained_model_name_or_path="microsoft/swin-tiny-patch4-window7-22
        num_labels= len(classes),
        id2label={idx: label for label, idx in class_to_idx.items()},
        label2id=class_to_idx,
        ignore_mismatched_sizes = True
    )
    return model

def collator(data, transform):
    images, labels = zip(*data)
    pixel_values = torch.stack([transform(image) for image in images])
    labels = torch.tensor([label for label in labels])
    return {'pixel_values': pixel_values, 'labels': labels} # dictionary 구조로 값 반환

def compute_metrics(eval_pred):
    metric = evaluate.load('f1')
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis = 1)
    macro_f1 = metric.compute(
        predictions = predictions, references = labels, average = 'macro'
    )
    return macro_f1

train_dataset = datasets.FashionMNIST(root='./datasets', download= True, tra

```

```

test_dataset = datasets.FashionMNIST(root='../datasets', download = True, tra

classes = train_dataset.classes
class_to_idx = train_dataset.class_to_idx

print(classes)
print(class_to_idx)

subset_train_dataset = subset_sampler(
    dataset = train_dataset, classes = train_dataset.classes, max_len = 1000
)
subset_test_dataset = subset_sampler(
    dataset = test_dataset, classes = test_dataset.classes, max_len = 100
)

image_processor = AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path = "microsoft/swin-tiny-patch4-window7-2
)

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Resize(
        size = (image_processor.size['height'], image_processor.size['width'])
    ),
    transforms.Lambda(lambda x: torch.cat([x, x, x], 0)),
    transforms.Normalize(mean = image_processor.image_mean,
        std = image_processor.image_std)
])

args = TrainingArguments(
    output_dir = '../models/Swin-FashionMNIST',
    save_strategy = 'epoch',
    eval_strategy = 'epoch',
    learning_rate = 1e-5,
    per_device_train_batch_size = 16,
    per_device_eval_batch_size = 16,
    num_train_epochs = 3,
    weight_decay = 0.001,

```

```

load_best_model_at_end = True,
metric_for_best_model = 'f1',
logging_dir = 'logs',
logging_steps = 125,
remove_unused_columns = False,
seed = 7
)

trainer = Trainer(
    model_init = lambda x: model_init(classes, class_to_idx),
    args = args,
    train_dataset = subset_train_dataset,
    eval_dataset = subset_test_dataset,
    data_collator = lambda x: collator(x, transform),
    compute_metrics = compute_metrics,
    tokenizer = image_processor
)

```

- ViT 모델 미세 조정 방식과 동일하게 훈련자 클래스(Trainer)를 통해 스윈 트랜스포머 모델의 학습을 수행한다. 사전 학습된 모델과 출력 디렉터리만 변경된다.
- 학습이 완료되면 Swin-FashionMNIST 디렉터리에 체크 포인트마다 학습결과, 성능 지표, 하이퍼 파라미터 등이 저장된다.

Epoch	Training Loss	Validation Loss	F1
1	0.3720	0.3046	0.8985
2	0.2579	0.2688	0.9138
3	0.2607	0.2416	0.9159

- 테스트 데이터셋으로 추론했을 때 스윈 트랜스포머 모델은 약 91.6%의 분류 정확도를 갖는다.
- ViT 모델의 학습 결과와 비교했을 때, F1 성능 지표는 거의 동일하지만, 검증 손실값이 0.2416으로 줄었다.

모델 성능 평가

- ViT 모델 평가와 같은 방식


```

import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

outputs = trainer.predict(subset_test_dataset)
print(outputs)

y_true = outputs.label_ids
y_pred = outputs.predictions.argmax(1)

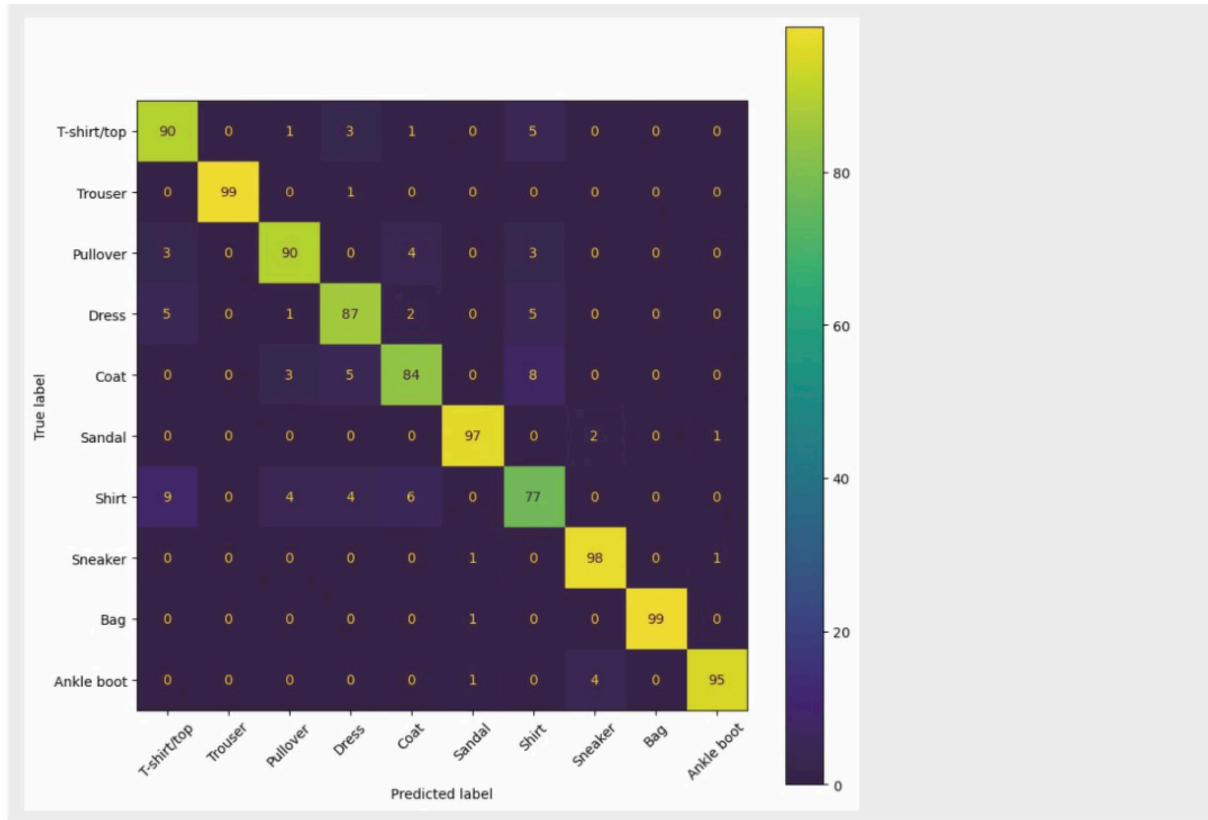
labels = list(classes)
matrix = confusion_matrix(y_true, y_pred)
display = ConfusionMatrixDisplay(confusion_matrix = matrix, display_labels =
_, ax = plt.subplots(figsize = (10, 10))
display.plot(xticks_rotation = 45, ax = ax)
plt.show()

```

```

# 생략
{
    'test_loss': 0.24168206751346588,
    'test_f1': 0.9159858553952347,
    'test_runtime': 4.3068,
    'test_samples_per_second': 58.048,
    'test_steps_per_second': 3.657
}
# 생략

```



- 스윈 트랜스포머 모델과 ViT 모델의 평가 결과를 비교해보자. 스윈 트랜스포머 모델이 셔츠 이미지를 더 높은 확률로 예측하며, 티셔츠, 코트로 오분류하는 문제가 완화된 것을 확인할 수 있다. 또, 스윈 트랜스포머 모델의 수렴 속도도 더 빠르다.

CvT

- **CvT(Convolutional Vision Transformer)**는 합성곱 신경망 구조에서 활용하는 계층 구조를 ViT에 적용한 모델이다.
- ViT의 셀프 어텐션은 이미지정보를 바탕으로 각 패치 간의 관계를 학습하지만, 모든 패치에 대해 동일한 크기와 간격을 사용하므로 이미지의 물체 크기나 해상도를 계층적으로 학습하기 어렵다.
- 이 문제를 해결하기 위해 CvT는 합성곱 신경망에서 사용하는 계층적 구조를 ViT에 적용해 이미지의 지역 특징과 전역 특징을 모두 활용해 비교적 적은 수의 매개 변수로 높은 성능을 보인다.
- CvT 구조는 합성곱 계층의 지역 특징을 추출하고 ViT의 셀프 어텐션 계층으로 전역 특징을 결합해 이미지를 처리한다. CvT 모델은 스윈 트랜스포머보다 더 적은 수의 매개 변수로도 비슷한 수준의 성능을 보인다.

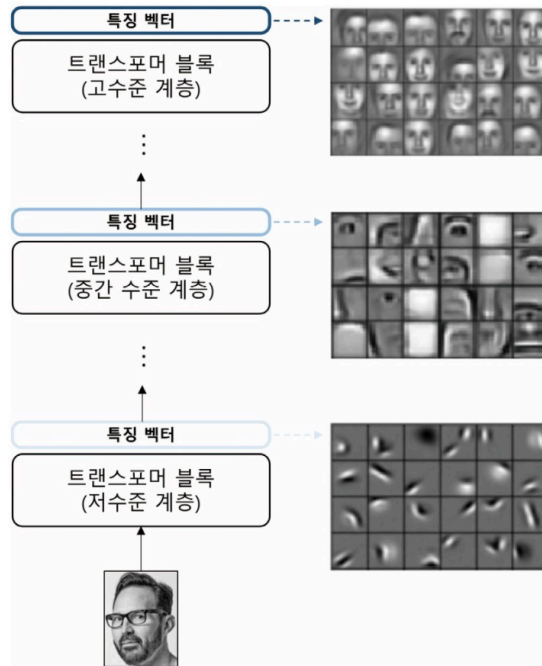


그림 10.14 CvT 모델의 계층적 특징 구조 예시

- 얼굴 이미지가 CvT 모델의 입력으로 사용된다면 저수준 계층에서 선 또는 점에 대한 특징을 표현하고 중간 수준에서는 눈이나 귀와 같은 특징을 표현하며, 고수준에서는 얼굴 구조에 대한 특징을 표현한다. 그러므로 CvT의 계층형 구조는 물체의 크기나 해상도를 더 잘 표현한다고 할 수 있다.

ViT와 CvT의 차이

- ViT는 이미지를 패치 단위로 분할하고, 이를 트랜스포머 모델의 입력으로 사용하는 방식을 사용해 패치를 **겹치지 않게 처리**하여 모델이 이미지의 전체 정보를 학습했다.
- 반면에 CvT는 이미지를 처리하기 전에 합성곱 연산으로 특징 맵을 추출한다. 이를 통해 유의미한 패턴을 추출하고 일정한 크기의 패치로 나눠 트랜스포머 모델의 입력으로 사용한다. 입력되는 **패치들은 겹쳐 학습**해서 토큰에 대한 위치 임베딩을 적용하지 않아도 모델의 성능 저하 없이 다양한 해상도를 가진 이미지를 처리할 수 있게 되었다.

모델	위치 임베딩	패치 임베딩	어텐션 임베딩	계층형 구조
ViT	O	겹치지 않는 선형 임베딩	선형 임베딩	X
CvT	X	겹치는 합성곱 임베딩 (overlapping)	합성곱 임베딩	O

CvT 특징

합성곱 토큰 임베딩

- 2D로 재구성된 토큰 맵에서 중첩 합성곱 연산을 수행하는 임베딩이다. ViT에 계층적인 합성곱 신경망 성질을 추가하기 위해 사용한다.
- 기존 트랜스포머 모델은 이미지 특징을 추출하기 위해 9개의 이미지 패치를 사용하고, 선형 임베딩을 통해 쿼리, 키, 밸류 임베딩으로 변환했다. 임베딩을 통해 셀프 어텐션을 계산하고 출력을 생성하므로 **이미지의 근본적인 2D 구조를 잃게 된다.**
- 반면, CvT 모델은 이미지의 2D 구조를 보존하면서도 셀프 어텐션을 계산하는 방식을 사용한다. 이미지 패치를 2D 합성곱 임베딩을 사용해 쿼리, 키, 밸류 임베딩으로 변환하고, 그 임베딩으로 셀프 어텐션을 계산하고 출력을 생성한다.
 - 쿼리: 3x3 커널 크기, 1 스트라이드 커널로 9 개의 이미지 패치 생성
 - 키, 밸류: 3x3 커널 크기, 2 스트라이드 커널로 4 개의 이미지 패치 생성

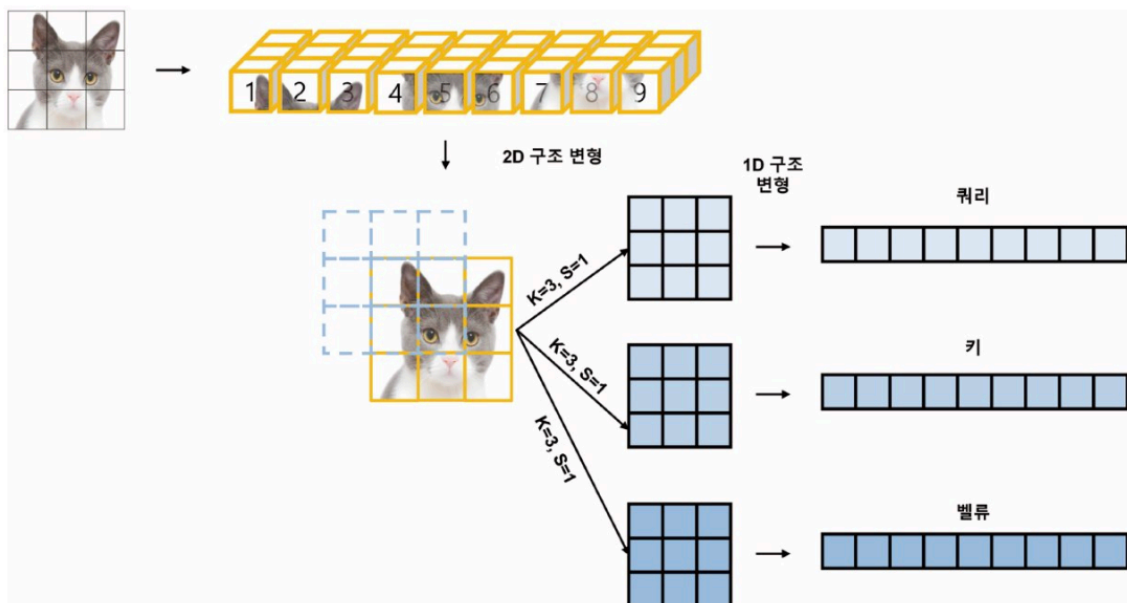


그림 10.15 2D 합성곱 기반의 어텐션 임베딩

- 길이가 다른 쿼리(9), 키(4), 밸류(4)를 사용하더라도 기존 셀프 어텐션의 결괏값 사이즈가 9x1로 동일하게 출력될 수 있다.

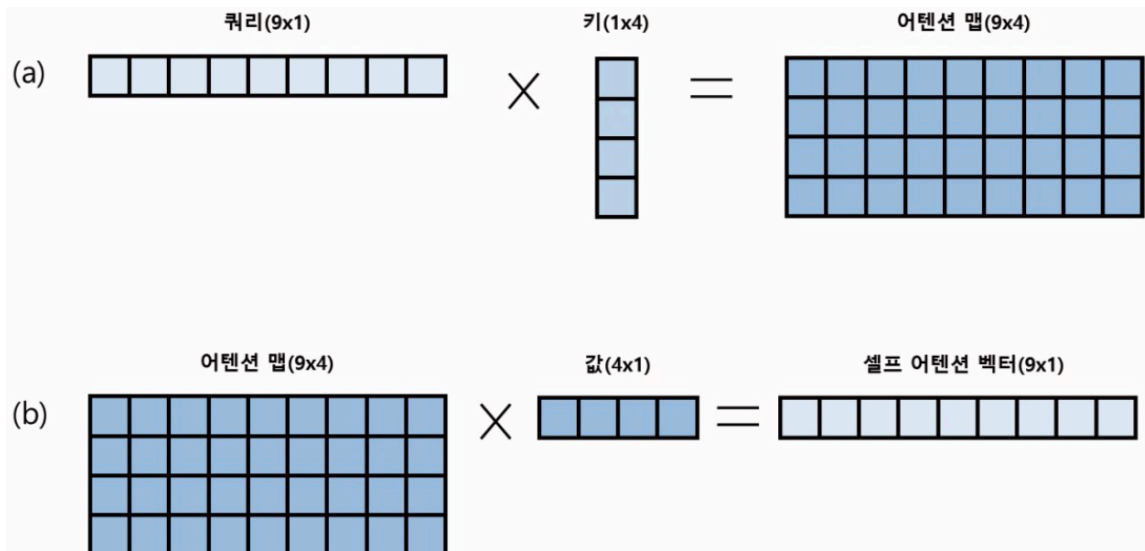


그림 10.17 축소된 셀프 어텐션 임베딩(열 × 행) 예시

- (a)에서 9개의 쿼리 패치 임베딩과 4 개의 키 패치 임베딩에 대한 중요도를 의미하는 **어텐션 맵**을 생성. 어텐션 맵이 생성되면 (b)와 같이 값 패치 임베딩으로 쿼리 어텐션 맵에 대한 **가중치**를 반영
- 셀프 어텐션 벡터는 입력 벡터 패치 길이 == 출력 벡터 패치 길이라서 계산해야하는 이미지 패치 개수가 줄어들어 연산량이 감소함
- CvT 모델은 트랜스포머 계층이 깊어질 수록 패치간 관계를 학습하는 길이를 축소시키는 대신, 벡터의 차원을 증가시켜 모델의 매개 변수 수를 대폭 감소시킴
 - 첫 번째 트랜스포머 블록의 이미지 패치 크기가 64x64, 임베딩 차원이 128이고 마지막 트랜스포머 블록의 이미지 패치 크기를 4배 감소시켜 8x8로 설정했다면, 기존 임베딩 차원도 4배 증가시켜 512 차원으로 증가시킨다.
 - 즉, 이미지 패치 크기가 줄어든 만큼 임베딩 차원을 늘리는 방법을 택해 네트워크 안정성을 향상시키고 모델 설계를 단순화한다.

실습

- mageNet-1k 데이터셋을 사용해 224x 224 이미지 크기로 학습된 microsoft/cwr-21 모델을 사용

10.20 CvT 모델 이미지 데이터 전처리

```
def subset_sampler(dataset, classes, max_len):
    target_idx = defaultdict(list)
    for idx, label in enumerate(dataset.targets):
        target_idx[int(label)].append(idx)
```

```

indices = list(
    chain.from_iterable(
        [target_idx[idx][:max_len] for idx in range(len(classes))]
    )
)
return Subset(datasets, indices)

train_dataset = datasets.FashionMNIST(root='../datasets', download= True, tra
test_dataset = datasets.FashionMNIST(root='../datasets', download = True, tra

# classes 메서드로 데이터셋에 포함된 클래스를 확인할 수 있음
# class_to_idx로 클래스 ID와 클래스가 매핑된 값 확인할 수 있음
classes = train_dataset.classes
class_to_idx = train_dataset.class_to_idx

# subset_sampler로 섭샘플링 (dataset, classes(클래스 목록), max_len(클래스별 최
subset_train_dataset = subset_sampler(
    dataset = train_dataset, classes = train_dataset.classes, max_len = 6000
)
subset_test_dataset = subset_sampler(
    dataset = test_dataset, classes = test_dataset.classes, max_len = 1000
)

import torch
from torchvision import transforms
from transformers import AutoImageProcessor

image_processor = AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path = 'microsoft/cvt-21'
)

transfrom = transforms.Compose([
    transforms.ToTensor(),
    transforms.Resize(
        size = (
            image_processor.size['shortes_edge'],
            image_processor.size['shortest_edge']

```

```

    )
),
transforms.Lambda(lambda x: torch.cat([x, x, x], 0)),
transforms.Normalize(mean = image_processor.image_mean,
                      std = image_processor.image_std)
])

from torch.utils.data import DataLoader

# ViT 모델에 맞는 입력 구조 변환 수행
def collator(data, transform):
    images, labels = zip(*data)
    pixel_values = torch.stack([transform(image) for image in images])
    labels = torch.tensor([label for label in labels])
    return {'pixel_values': pixel_values, 'labels': labels}
# dictionary 구조로 값 반환

train_dataloader = DataLoader(
    subset_train_dataset, batch_size = 32, shuffle = True,
    collate_fn = lambda x: collator(x, transform), drop_last = True
)
valid_dataloader = DataLoader(
    subset_test_dataset, batch_size = 4, shuffle = True,
    collate_fn = lambda x: collator(x, transform), drop_last = True
)

batch = next(iter(train_dataloader))
for key, value in batch.items():
    print(f'{key}: {value.shape}')

```

- ViT와 스윈 트랜스포머 모델은 이미지 크기 전처리 과정에서 높이와 너비를 이미지 프로세서에서 불러와 적용했지만, CvT 모델은 이미지 크기를 조절할 때 `shortest_edge` 키를 사용해 전처리한다.
 - 이는 이미지의 너비나 높이 중 더 작은 값을 의미한다. microsoft/cvt-21 모델은 모든 입력 이미지 크기를 가장 짧은 이미지 길이로 정규화해 사용하므로 높이나 너비에 대한 키 대신 `shortest_edge` 를 사용한다.

10.21 사전 학습된 CvT 모델

```

from transformers import CvTForImageClassification

model = CvTForImageClassification.from_pretrained(
    pretrained_model_name_or_path = 'microsoft/cvt-21',
    num_labels = len(train_dataset.classes),
    id2label = {idx: label for label, idx in train_dataset.class_to_idx.items()},
    label2id = train_dataset.class_to_idx,
    ignore_mismatched_sizes = True
)

for main_name, main_module in model.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print('L', sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("  L", ssub_name)
            for sssub_name, sssub_module in ssub_module.named_children():
                print("    L", sssub_name)

```

```

cvt
├── encoder
│   ├── stages
│   │   ├── 0
│   │   ├── 1
│   │   └── 2
│   ├── layernorm
│   └── classifier

```

- CvT 모델은 크게 세 개의 스테이지로 구성된 cvt와 계층 정규화, 분류기로 구성되어 있다. 스윈 트랜스포머와 달리 분류기와 풀링 계층이 없다. CvT 모델은 이미 합성곱 계층을 활용해 지역 특징 정보와 전역 정보를 파악하기 때문에 풀링을 적용하지 않는다.

10.22 CvT 모델의 스테이지 구조

- 스테이지별 이미지 크기가 4배, 8배, 16배 감소되며, 반비례하게 임베딩 차원이 커진다.

```

stages = model.cvt.encoder.stages
print(stages[0])

```



```

CvtStage(
  (embedding): CvtEmbeddings(
    (convolution_embeddings): CvtConvEmbeddings(
      (projection): Conv2d(3, 64, kernel_size=(7, 7), stride=(4, 4), padding=(2, 2))
      (normalization): LayerNorm((64,)), eps=1e-05, elementwise_affine=True)
    )
    (dropout): Dropout(p=0.0, inplace=False)
  )
  (layers): Sequential(
    (0): CvtLayer(
      (attention): CvtAttention(
        (attention): CvtSelfAttention(
          (convolution_projection_query): CvtSelfAttentionProjection(
            (convolution_projection): CvtSelfAttentionConvProjection(
              (convolution): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
groups=64, bias=False)
              (normalization): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
            (linear_projection): CvtSelfAttentionLinearProjection()
          )
          (convolution_projection_key): CvtSelfAttentionProjection(
            (convolution_projection): CvtSelfAttentionConvProjection(
              (convolution): Conv2d(64, 64, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
groups=64, bias=False)
              (normalization): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
            (linear_projection): CvtSelfAttentionLinearProjection()
          )
          (convolution_projection_value): CvtSelfAttentionProjection(
            (convolution_projection): CvtSelfAttentionConvProjection(
              (convolution): Conv2d(64, 64, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
groups=64, bias=False)
              (normalization): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
          )
        )
      )
    )
  )
)

```

```

        (linear_projection): CvtSelfAttentionLinearProjection()
    )
    ...
)
...
)
...
)
)
)
)

```

- 각 스테이지는 크게 **CvTEmbeddings** 와 **CvTLayer** 로 구성된다.
- **CvTEmbeddings** 는 입력 이미지에 대한 2D 합성곱 연산과 계층 정규화로 이뤄져 있으며 이미지를 패치 단위로 잘라 임베딩한다.
- 입력 이미지는 첫 번째 스테이지의 **CvTEmbeddings** 에 적용되는데, 7x7 커널 크기, 4 스트라이드, 2 패딩 크기로 합성곱 연산을 적용하면 $(224-7+2 \times 2)/4 + 1 = 56.25$ 에서 버림처리를 한다.
- 입력 이미지를 224 x 224 크기 → 56 x 56 크기의 텐서로 변환한다. 이때, 매개 변수가 축소된 어텐션 임베딩을 계산하기 위해 키와 밸류 임베딩 간격을 2로 설정해 차원을 축소한다.
- **CvTLayer** 에서 3136(56 x 56)개의 이미지 패치에 대한 셀프 어텐션을 수행한다.

10.23 셀프 어텐션 적용

```
batch = next(iter(train_dataloader))
print(" 이미지 차원 :", batch["pixel_values"].shape)

patch_emb_output = stages[0].embedding(batch["pixel_values"])
print(" 패치 임베딩 차원 :", patch_emb_output.shape)

batch_size, num_channels, height, width = patch_emb_output.shape
hidden_state = patch_emb_output.view(batch_size, num_channels, height * width)
print('셀프 어텐션 입력 차원: ', hidden_state.shape)

attention_output = stages[0].layers[0].attention.attention(hidden_state, height, width)
print(" 셀프 어텐션 출력 차원 : ", attention_output.shape)
```

```
이미지 차원 : torch.Size([32, 3, 224, 224])
패치 임베딩 차원 : torch.Size([32, 64, 56, 56])
셀프 어텐션 입력 차원 : torch.Size([32, 3136, 64])
셀프 어텐션 출력 차원 : torch.Size([32, 3136, 64])
```

- 합성곱 계층을 통해 계산된 패치 임베딩은 가로와 세로 크기를 재정렬해 `CvtLayer`에 입력되고, `CvtLayer`의 `attention.attention` 계층을 통해 셀프 어텐션이 수행된다. 출력 결과에서 입력 차원과 출력 차원의 형태가 동일한 것을 확인할 수 있다.

10.24 CvT 모델 학습

```
from transformers import CvtForImageClassification

def subset_sampler(dataset, classes, max_len):
    target_idx = defaultdict(list)
    for idx, label in enumerate(dataset.targets):
        target_idx[int(label)].append(idx)

    indices = list(
        chain.from_iterable(
            [target_idx[idx][:max_len] for idx in range(len(classes))]
        )
    )
    return Subset(dataset, indices)

def model_init(classes, class_to_idx):
    model = CvtForImageClassification.from_pretrained(
        pretrained_model_name_or_path="microsoft/cvt-21",
        num_labels=len(classes),
        id2label={idx: label for label, idx in class_to_idx.items()},
        label2id=class_to_idx,
        ignore_mismatched_sizes = True
    )
    return model

def collator(data, transform):
    images, labels = zip(*data)
    pixel_values = torch.stack([transform(image) for image in images])
    labels = torch.tensor([label for label in labels])
```

```

return {'pixel_values': pixel_values, 'labels': labels} # dictionary 구조로 값 반환

def compute_metrics(eval_pred):
    metric = evaluate.load('f1')
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis = 1)
    macro_f1 = metric.compute(
        predictions = predictions, references = labels, average = 'macro'
    )
    return macro_f1

train_dataset = datasets.FashionMNIST(root='./datasets', download= True, tra
test_dataset = datasets.FashionMNIST(root='./datasets', download = True, tra

classes = train_dataset.classes
class_to_idx = train_dataset.class_to_idx

subset_train_dataset = subset_sampler(
    dataset = train_dataset, classes = train_dataset.classes, max_len = 1000
)
subset_test_dataset = subset_sampler(
    dataset = test_dataset, classes = test_dataset.classes, max_len = 100
)

image_processor = AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path = "microsoft/cvt-21"
)

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Resize(
        size = (image_processor.size['shortest_edge'], image_processor.size['sh
    ),
    transforms.Lambda(lambda x: torch.cat([x, x, x], 0)),
    transforms.Normalize(mean = image_processor.image_mean,
        std = image_processor.image_std)
])

```

```

args = TrainingArguments(
    output_dir = './models/CvT-FashionMNIST',
    save_strategy = 'epoch',
    eval_strategy = 'epoch',
    learning_rate = 1e-5,
    per_device_train_batch_size = 16,
    per_device_eval_batch_size = 16,
    num_train_epochs = 3,
    weight_decay = 0.001,
    load_best_model_at_end = True,
    metric_for_best_model = 'f1',
    logging_dir = 'logs',
    logging_steps = 125,
    remove_unused_columns = False,
    seed = 7
)

trainer = Trainer(
    model_init = lambda x: model_init(classes, class_to_idx),
    args = args,
    train_dataset = subset_train_dataset,
    eval_dataset = subset_test_dataset,
    data_collator = lambda x: collator(x, transform),
    compute_metrics = compute_metrics,
    tokenizer = image_processor
)

```

- CvT-FashionMNIST 폴더에 체크포인트마다 학습 결과, 성능 지표, 하이퍼 파라미터 등이 저장된다.

Epoch	Training Loss	Validation Loss	F1
1	0.7737	0.3269	0.8914
2	0.7008	0.2596	0.9172
3	0.6438	0.2460	0.9173

- 테스트 데이터셋으로 추론했을 때 CvT 모델은 91.3%의 분류 정확도를 갖는다. 스원 트랜스포머 모델과 CvT 모델의 F1 점수 결과를 비교했을 때 0.9159에서 0.9173으로 소폭 개선됐다.

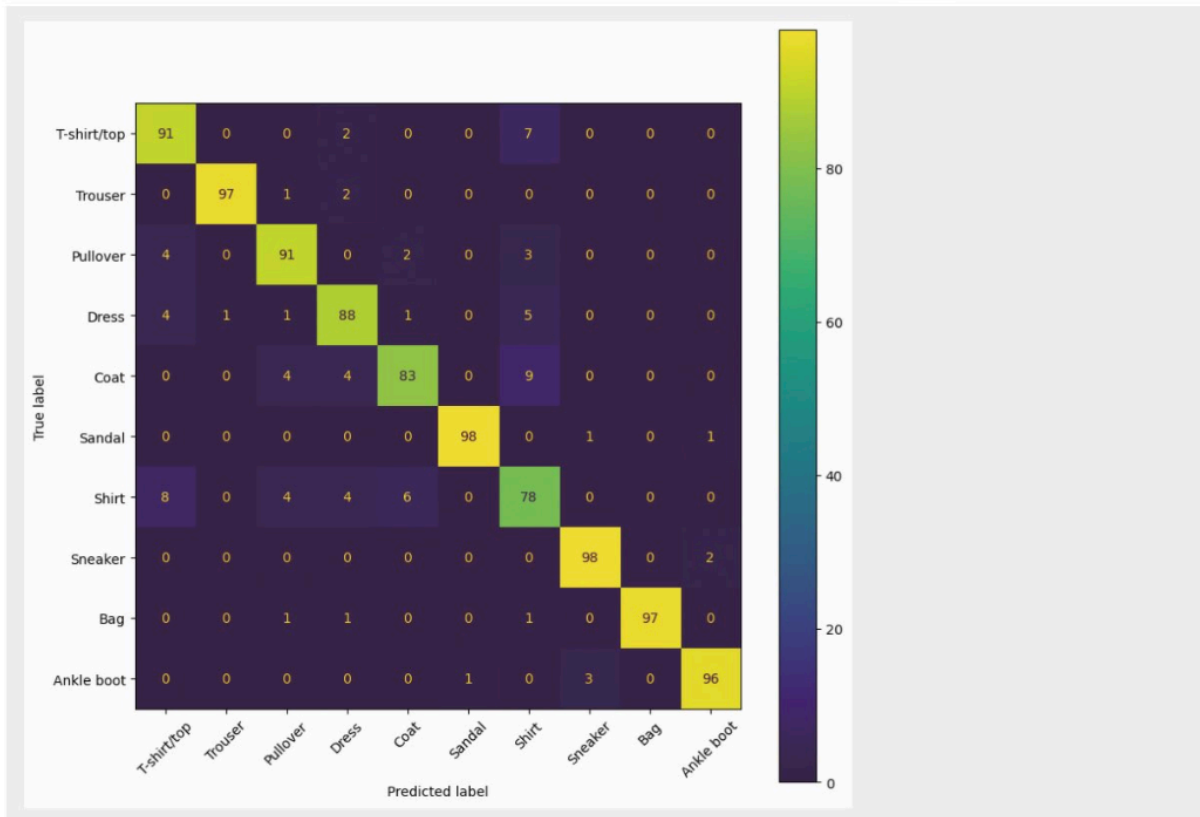
```
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

outputs = trainer.predict(subset_test_dataset)
print(outputs)

y_true = outputs.label_ids
y_pred = outputs.predictions.argmax(1)

labels = list(classes)
matrix = confusion_matrix(y_true, y_pred)
display = ConfusionMatrixDisplay(confusion_matrix = matrix, display_labels =
_, ax = plt.subplots(figsize = (10, 10))
display.plot(xticks_rotation = 45, ax = ax)
plt.show()
```

```
# 생략
{
    'test_loss': 0.24601557850837708,
    'test_f1': 0.917351060925814,
    'test_runtime': 5.583,
    'test_samples_per_second': 44.778,
    'test_steps_per_second': 2.821
}
# 생략
```



- CvT 모델과 ViT 모델을 비교하면 셔츠 카테고리 성능이 개선되었으며, 스원 트랜스포머와 비슷한 성능을 보인다.

최종 결론

- 이 챕터에서 소개된 3개의 모델 크기 및 성능을 종합하면 다음과 같다.

	ViT	스원 트랜스포머	CvT
모델 크기	327,325 MB	105,227 MB	120,791 MB
F1-점수	0.9231	0.9159	0.9173
초당 처리 수	29,636 eval/s	58,048 eval/s	44,778 eval/s

- FashionMNIST 데이터세트에서 비전 트랜스포머 계열 모델은 작은 크기의 학습 데이터세트에서도 높은 정확도와 우수한 성능을 보인다. 가장 일반화 성능이 좋았던 모델은 CVT 모델, 모델 크기 대비 가장 우수한 성능은 스원 트랜스포머 모델, F1- 점수가 가장 높은 모델은 ViT 모델이다. 각각의 모델을 풀어야하는 문제와 사용처에 따라 적절히 선택한다.